

```
In [2]: 1 import array
        2 import cmath
        3 import collections
        4 import copy
        5 import csv
        6 import datetime
        7 import decimal
        8 import functools
        9 import importlib
       10 import logging
       11 import math
       12 import pprint
       13 import random
       14 import re
       15 import statistics
       16 import string
       17 import sys
       18 import time
       19 import timeit
       20
       21 import pandas
```

```
In [2]: 1 num1 = 10_00_000
        2 print(num1)
```

```
1000000
```

```
In [3]: 1 print(format(0.1, ".25f"))
```

```
0.10000000000000000055511151
```

```
In [4]: 1 print(format(0.125, ".25f"))
```

```
0.12500000000000000000000000
```

```
In [5]: 1 print(0.1 + 0.1 + 0.1 == 0.3)
```

```
False
```

```
In [6]: 1 print(format(0.3, ".25f"))
```

```
0.2999999999999999888977698
```

```
In [7]: 1 print(format(0.2, ".25f"))
```

```
0.200000000000000011022302
```

```
In [8]: 1 print(abs((0.1 + 0.1 + 0.1) - 0.3) < 0.001)
```

```
True
```

```
In [9]: 1 print((0.125).as_integer_ratio())
```

(1, 8)

```
In [10]: 1 print((10).      (25))
```

35

```
In [11]: 1 a = float(10)
        2 print(a)
```

10.0

```
In [12]: 1 c = (-4) ** 0.5
        2 print(c)
```

(1.2246467991473532e-16+2j)

```
In [13]: 1 print(type(c))
```

<class 'complex'>

```
In [14]: 1 print(c.real)
```

1.2246467991473532e-16

```
In [15]: 1 print(c.imag)
```

2.0

```
In [16]: 1 c = 10 + 2j
        2 print(type(c))
```

<class 'complex'>

```
In [17]: 1 print(c)
```

(10+2j)

```
In [18]: 1 try:
        2     print("2" < 2)
        3 except TypeError as ex:
        4     print(type(ex).      )
```

TypeError

```
In [19]: 1 print("2" == 2)
```

False

```
In [20]: 1 a = 0
        2 b = 1
        3 c = 500
        4 d = 0
        5 e = 1
        6 f = 500
        7 print(id(a), id(b), id(c), id(d), id(e), id(f))
```

11370064 11370096 128573173327504 11370064 11370096 128573173326608

```
In [21]: 1 print(0.1 * 3 - 0.3 < abs(1e-6))
```

True

```
In [22]: 1 a = 257
        2 print(a is 257)
```

False

<>:2: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?

<>:2: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?

/tmp/ipykernel_18021/1958631963.py:2: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
print(a is 257)

```
In [23]: 1 a = 10
        2 b = 0
        3 print(b != 0 and a / b)
```

False

```
In [24]: 1 a = 1e-6
        2 b = 2e-6
        3 tol = 1e-7
        4 print(abs(a - b) <= tol)
```

False

```
In [25]: 1 elapsed = 7835
        2 hours = elapsed // 3600
        3 minutes = (elapsed - (hours * 3600)) // 60
        4 seconds = elapsed - ((hours * 3600) + (minutes * 60))
        5 print(hours, minutes, seconds)
```

2 10 35

```
In [26]: 1 elapsed = 7835
        2 hours = elapsed // (60 * 60)
        3 minutes = (elapsed % (60 * 60)) // 60
        4 seconds = (elapsed % (60 * 60)) % 60
        5 print(hours, minutes, seconds)
```

2 10 35

```
In [27]: 1 a = None
        2 if a == None:
        3     a = "N/A"
        4 print(a)
```

N/A

```
In [28]: 1 a = None
        2 a = "N/A" if a is None else a
        3 print(a)
```

N/A

In [29]:

```
1 score = 700
2 rating = "Invalid"
3 if score < 0:
4     rating = "Negative"
5 elif score < 580:
6     rating = "Poor"
7 elif score < 670:
8     rating = "Fair"
9 elif score < 740:
10    rating = "Good"
11 elif score < 800:
12    rating = "Very Good"
13 elif score < 850:
14    rating = "Excellent"
15 print(rating)
```

Good

In [30]:

```
1 elapsed = 1000000
2 magnitude = "Invalid"
3 if elapsed < 60:
4     magnitude = "seconds"
5 elif elapsed < (60 * 60):
6     magnitude = "minutes"
7 elif elapsed < (60 * 60 * 24):
8     magnitude = "hours"
9 elif elapsed < (60 * 60 * 24 * 7):
10    magnitude = "days"
11 elif elapsed > (60 * 60 * 24 * 7):
12    magnitude = "weeks"
13 print(magnitude)
```

weeks

In [31]:

```
1 t = [1, 2], 3, 4
2 l = list(t)
3 print(l)
```

[[1, 2], 3, 4]

In [32]:

```
1 l[0][1] = 20
2 print(l)
```

[[1, 20], 3, 4]

In [33]:

```
1 print(t)
```

([1, 20], 3, 4)

In [34]:

```
1 for i in t:
2     print(id(i))
```

```
128573172499456
11370160
11370192
```

```
In [35]: 1 for i in l:
          2     print(id(i))
```

```
128573172499456
11370160
11370192
```

```
In [36]: 1 for i, j in zip(l, t):
          2     print(i is j)
```

```
True
True
True
```

```
In [37]: 1 s = "ABCDEFGHIIJ"
          2 print(s[3:6:-1])
```

```
In [38]: 1 print(s[3:5 + 1])
```

```
DEF
```

```
In [39]: 1 m1 = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
          2 m2 = m1[0:2]
          3 print(m1[0] is m2[1])
```

```
False
```

```
In [40]: 1 m1[0] = "Python"
          2 print(m1)
```

```
['Python', [0, 0, 0], [0, 0, 0]]
```

```
In [41]: 1 print(m2)
```

```
[[0, 0, 0], [0, 0, 0]]
```

```
In [42]: 1 m1[1][0] = 100
```

```
In [43]: 1 print(m1)
```

```
['Python', [100, 0, 0], [0, 0, 0]]
```

```
In [44]: 1 print(m2)
```

```
[[0, 0, 0], [100, 0, 0]]
```

```
In [45]: 1 print("abcdef"[-1:-4])
```

```
In [46]: 1 print("abcdef"[1:4:-1])
```

```
In [47]: 1 print("racecar" == "racecar"[::-1])
```

```
True
```

```
In [48]: 1 list1 = [0, 1, 2, 3, 4]
        2 list1[0:3] = "abc"
        3 print(list1)
```

```
['a', 'b', 'c', 3, 4]
```

```
In [49]: 1 list1[0:3] = ["a", "b", "c"]
        2 print(list1)
```

```
['a', 'b', 'c', 3, 4]
```

```
In [50]: 1 list1[0:3] = ["a", "b"]
        2 print(list1)
```

```
['a', 'b', 3, 4]
```

```
In [51]: 1 del list1[2]
        2 print(list1)
```

```
['a', 'b', 4]
```

```
In [52]: 1 del list1[0:2]
        2 print(list1)
```

```
[4]
```

```
In [53]: 1 list1.extend("ABC")
        2 print(list1)
```

```
[4, 'A', 'B', 'C']
```

```
In [54]: 1 list1.insert(0, "YO!")
        2 print(list1)
```

```
['YO!', 4, 'A', 'B', 'C']
```

```
In [55]: 1 list1[:-3:-1] = "cb"
        2 print(list1)
```

```
['YO!', 4, 'A', 'b', 'c']
```

```
In [56]: 1 l = []
        2 print(timeit.timeit("l.append(1)", globals = globals(), number = 100_000))
```

```
0.002112314999976661
```

```
In [57]: 1 print(len(l))
```

```
100000
```

```
In [58]: 1 list2 = []
        2 print(timeit.timeit("list2.insert(0, 1)", globals = globals(), number =
```

```
100_000))
```

```
1.5377864980000027
```

```
In [59]: 1 print(len(list2))
```

```
100000
```

```
In [60]: 1 print(1.249788 / 0.0018043)
```

```
692.6719503408524
```

```
In [61]: 1 list2 = []  
2 print(timeit.timeit("list2.insert(len(list2), 1)", globals = globals(), number  
= 100_000))
```

```
0.008461738999983481
```

```
In [62]: 1 print(0.0057634 / 0.0018043)
```

```
3.194258161059691
```

```
In [63]: 1 l1 = [[0, 1], 2, "3"]  
2 l2 = l1[:]  
3 for i in l1:  
4     print(id(i))
```

```
128573172569024  
11370128  
11434576
```

```
In [64]: 1 for i in l2:  
2     print(id(i))
```

```
128573172569024  
11370128  
11434576
```

```
In [65]: 1 l2[0][0] = "0"  
2 for i in l1:  
3     print(id(i))
```

```
128573172569024  
11370128  
11434576
```

```
In [66]: 1 for i in l2:  
2     print(id(i))
```

```
128573172569024  
11370128  
11434576
```

```
In [67]: 1 for i in l1:  
2     print(i)
```

```
['0', 1]  
2  
3
```

```
In [68]: 1 for i in l2:  
        2     print(i)
```

```
['0', 1]  
2  
3
```

```
In [69]: 1 l2[1] = "22"  
        2 for i in l1:  
        3     print(id(i))
```

```
128573172569024  
11370128  
11434576
```

```
In [70]: 1 for i in l2:  
        2     print(id(i))
```

```
128573172569024  
128574645937104  
11434576
```

```
In [71]: 1 for i in l1:  
        2     print(i)
```

```
['0', 1]  
2  
3
```

```
In [72]: 1 for i in l2:  
        2     print(i)
```

```
['0', 1]  
22  
3
```

```
In [73]: 1 l1 = [['0', 1], 2, "3"]  
        2 l2 = l1[:]  
        3 l2[0] = ["0", 1]  
        4 for i in l1:  
        5     print(id(i))
```

```
128573172599040  
11370128  
11434576
```

```
In [74]: 1 for i in l2:  
        2     print(id(i))
```

```
128573172596864  
11370128  
11434576
```

```
In [75]: 1 for i in l1:
         2     print(i)
```

```
[0, 1]
2
3
```

```
In [76]: 1 for i in l2:
         2     print(i)
```

```
['0', 1]
2
3
```

```
In [77]: 1 l1 = [[0, 1], 2, "3"]
         2 l2 = l1[:]
         3 print(id(l1), id(l2))
```

```
128573172599104 128573172598912
```

```
In [78]: 1 for i, j in zip(l1, l2):
         2     print(id(i), id(j))
```

```
128573172598848 128573172598848
11370128 11370128
11434576 11434576
```

```
In [79]: 1 for i, j in zip(l1[0], l2[0]):
         2     print(id(i), id(j))
```

```
11370064 11370064
11370096 11370096
```

```
In [80]: 1 l1 = [[0, 1], 2, "3"]
         2 l2 = l1.copy()
         3 print(id(l1), id(l2))
```

```
128573172597248 128573172597312
```

```
In [81]: 1 for i, j in zip(l1, l2):
         2     print(id(i), id(j))
```

```
128573172601472 128573172601472
11370128 11370128
11434576 11434576
```

```
In [82]: 1 for i, j in zip(l1[0], l2[0]):
         2     print(id(i), id(j))
```

```
11370064 11370064
11370096 11370096
```

```
In [83]: 1 l1 = [[0, 1], 2, "3"]
         2 l2 = copy.deepcopy(l1)
         3 print(id(l1), id(l2))
```

```
128574051537216 128573172602368
```

```
In [84]: 1 for i, j in zip(l1, l2):  
        2     print(id(i), id(j))
```

```
128573172600640 128573172437184  
11370128 11370128  
11434576 11434576
```

```
In [85]: 1 for i, j in zip(l1[0], l2[0]):  
        2     print(id(i), id(j))
```

```
11370064 11370064  
11370096 11370096
```

```
In [86]: 1 t1 = (0, "1", "23")  
        2 t2 = t1[:]  
        3 print(t1 is t2)
```

```
True
```

```
In [87]: 1 a, b, c, d = "abcd"  
        2 print(a, b, c, d)
```

```
a b c d
```

```
In [88]: 1 s = "FfEeDdCcBbAa"  
        2 lowerLetters = s[::-2]  
        3 upperLetters = s[-2::-2]  
        4 print(lowerLetters)
```

```
abcdef
```

```
In [89]: 1 print(upperLetters)
```

```
ABCDEF
```

```
In [90]: 1 t1 = 1, 2, 3, 4, 5, 6  
        2 t2 = 7, 8, 9, 10  
        3 t3 = 11, 12, 13, 14, 15, 16, 17  
        4 t = t1 + t2 + t3  
        5 l = list(t)  
        6 l[::2] = [0] * len(l[::2])  
        7 print(tuple(l))
```

```
(0, 2, 0, 4, 0, 6, 0, 8, 0, 10, 0, 12, 0, 14, 0, 16, 0)
```

```
In [91]: 1 m = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]  
        2 m[0][0] = 1  
        3 m[1][1] = 1  
        4 m[2][2] = 1  
        5 print(m)
```

```
[[1, 0, 0], [0, 1, 0], [0, 0, 1]]
```

```
In [92]: 1 m = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
2 m1 = copy.deepcopy(m)
3 m1[0][0] = 1
4 m1[1][1] = 1
5 m1[2][2] = 1
6 print(m1)
```

```
[[1, 0, 0], [0, 1, 0], [0, 0, 1]]
```

```
In [93]: 1 print(m)
2 data = [(100, "USD", "EUR", 0.83), (100, "USD", "CAD", 1.27), (100, "CAD",
3 "EUR", 0.65)]
4 USDAmountInEUR = data[0][0] * data[0][3]
5 print(str(data[0][0]) + " " + data[0][1] + " = " + str(USDAmountInEUR) + " " +
str(data[0][2]))
```

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
100 USD = 83.0 EUR
```

```
In [94]: 1 USDAmountInCAD = data[1][0] * data[1][3]
2 print(str(data[1][0]) + " " + data[1][1] + " = " + str(USDAmountInCAD) + " " +
str(data[1][2]))
```

```
100 USD = 127.0 CAD
```

```
In [95]: 1 CADAmountInEUR = data[2][0] * data[2][3]
2 print(str(data[2][0]) + " " + data[2][1] + " = " + str(CADAmountInEUR) + " " +
str(data[2][2]))
```

```
100 CAD = 65.0 EUR
```

```
In [96]: 1 print(ord("A"))
```

```
65
```

```
In [97]: 1 print("\N{SNAKE}")
```

```
🐍
```

```
In [98]: 1 print(ord("\N{SNAKE}"))
```

```
128013
```

```
In [99]: 1 print(hex(ord("\N{SNAKE}"))) 
```

```
0x1f40d
```

```
In [100]: 1 print("\u03b1")
```

```
α
```

```
In [101]: 1 print("\U0001f40d")
```

```
🐍
```

```
In [102]: 1 print(int("1f40d", 16))
```

```
128013
```

```
In [103]: 1 print("\N{SNAKE} is a snake")
```

```
🐍 is a snake
```

```
In [104]: 1 print(int("7ff", 16))
```

```
2047
```

```
In [105]: 1 print(int("ffff", 16))
```

```
65535
```

```
In [106]: 1 print(int("10ffff", 16))
```

```
1114111
```

```
In [107]: 1 print(bin(2047))
```

```
0b1111111111
```

```
In [108]: 1 print(len(bin(2047)))
```

```
13
```

```
In [109]: 1 print(bin(65535))
```

```
0b1111111111111111
```

```
In [110]: 1 print(len(bin(65535)))
```

```
18
```

```
In [111]: 1 print(bin(1114111))
```

```
0b10000111111111111111
```

```
In [112]: 1 print(len(bin(1114111)))
```

```
23
```

```
In [113]: 1 print(bin(128013))
```

```
0b11111010000001101
```

```
In [114]: 1 print("\n{snake}")
```

```
{snake}
```

```
In [115]: 1 a = True  
2 b = True  
3 print(a is b)
```

```
True
```

```
In [116]: 1 a = False
          2 b = False
          3 print(a is b)
```

True

```
In [117]: 1 a = True
          2 print(id(a))
```

11120192

```
In [118]: 1 a = False
          2 print(id(a))
```

11045056

```
In [119]: 1 b = False
          2 print(id(b))
```

11045056

```
In [120]: 1 ?
```

Docstring:

Return the lowest index in S where substring sub is found, such that sub is contained within S[start:end].

Optional arguments start and end are interpreted as in slice notation.
Return -1 on failure.

Type: method_descriptor

```
In [121]: 1 print(f"1 + 1 = {1 + 1}")
```

1 + 1 = 2

```
In [122]: 1 bid = 1.5760
          2 ask = 1.5763
          3 print("bid: {}, ask: {}, spread: {}".format(bid, ask, ask - bid))
```

bid: 1.576, ask: 1.5763, spread: 0.000299999999999996696

```
In [123]: 1 print("bid: {:.4f}, ask: {:.4f}, spread: {:.4f}".format(bid, ask, ask - bid))
```

bid: 1.5760, ask: 1.5763, spread: 0.0003

```
In [124]: 1 print(f"bid: {bid:.4f}, ask: {ask:.4f}, spread: {ask - bid:.4f}")
```

bid: 1.5760, ask: 1.5763, spread: 0.0003

```
In [125]: 1 print(f"bid: {bid:.4f}, ask: {ask:.4f}, spread: {(ask - bid):.4f}")
```

bid: 1.5760, ask: 1.5763, spread: 0.0003

```
In [126]: 1 string1 = "β, β, #"
          2 string1 = "Π, ú, θ, ω, ν"
```

```
3 chars = string1.split(", ")
4 print(chars)
```

```
['Π', 'ó', 'θ', 'ω', 'ν']
```

```
In [127]: 1 list1 = []
          2 for c in chars:
          3     list1.append(hex(ord(c)))
          4 print(list1)
```

```
['0x3a0', '0x3cd', '0x3b8', '0x3c9', '0x3bd']
```

```
In [128]: 1 list1 = []
          2 for c in chars:
          3     list1.append(c.upper())
          4 print(list1)
```

```
['Π', 'Υ', 'Θ', 'Ω', 'Ν']
```

```
In [129]: 1 list1 = []
          2 for c in chars:
          3     list1.append(c.lower())
          4 print(list1)
```

```
['π', 'ó', 'θ', 'ω', 'ν']
```

```
In [130]: 1 a = 42
          2 print(f"The number {a} is not even") if a % 2 else print(f"The number {a} is even")
```

```
The number 42 is even
```

```
In [131]: 1 a = 41
          2 print(f"The number {a} is not even") if a % 2 else print(f"The number {a} is even")
```

```
The number 41 is not even
```

```
In [132]: 1 a = 42
          2 print("The number {} is not even".format(a)) if a % 2 else print("The number {} is even".format(a))
```

```
The number 42 is even
```

```
In [133]: 1 a = 41
          2 print("The number {} is not even".format(a)) if a % 2 else print("The number {} is even".format(a))
```

```
The number 41 is not even
```

```
In [134]: 1 a = 0.3
          2 b = 0.1
          3 print(f"{a} / {b} = {a / b:.4f}")
```

```
0.3 / 0.1 = 3.0000
```

```
In [135]: 1 a = 42
          2 qualifier = "is not" if a % 2 else "is"
          3 print(f"The number {a} {qualifier} even")
```

The number 42 is even

```
In [136]: 1 a = 421
          2 qualifier = "is not" if a % 2 else "is"
          3 print(f"The number {a} {qualifier} even")
```

The number 421 is not even

```
In [137]: 1 print(range(10))
```

range(0, 10)

```
In [138]: 1 print(type(range(10)))
```

<class 'range'>

```
In [139]: 1 for i in range(3):
          2     print(i)
          3     print(i)
```

0
1
2
2

```
In [140]: 1 for i in "abc":
          2     pass
          3     print(i)
```

c

```
In [141]: 1 raggedArray = [[1, 2], [3, 4, 5, 6], [7, 8, 9], [10]]
          2 for r in range(len(raggedArray)):
          3     for c in range(len(raggedArray[r])):
          4         print(f"[{r}, {c}] = {raggedArray[r][c]}")
          5     print("-" * 5)
```

[0, 0] = 1
[0, 1] = 2

[1, 0] = 3
[1, 1] = 4
[1, 2] = 5
[1, 3] = 6

[2, 0] = 7
[2, 1] = 8
[2, 2] = 9

[3, 0] = 10

```
In [142]: 1 identityMatrix = []
2 size = 10
3 for r in range(size):
4     row = []
5     for c in range(size):
6         if r == c:
7             row.append(1)
8         else:
9             row.append(0)
10    identityMatrix.append(row)
11    identityMatrix
```

```
Out[142]: [[1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 1, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 1]]
```

```
In [143]: 1 raggedArray = [[1, 2], [3, 4, 5, 6], [7, 8, 9], [10]]
2 for i, r in enumerate(raggedArray):
3     for j, e in enumerate(raggedArray[i]): # e for element, cuz it's not a
4         column technically
5         print(f"[{i}, {j}] = {raggedArray[i][j]}")
6         print("-" * 8)
```

```
[0, 0] = 1
[0, 1] = 2
-----
[1, 0] = 3
[1, 1] = 4
[1, 2] = 5
[1, 3] = 6
-----
[2, 0] = 7
[2, 1] = 8
[2, 2] = 9
-----
[3, 0] = 10
-----
```

```
In [144]: 1 print(f"{0.1:.4f}")
```

```
0.1000
```

```
In [145]: 1 print("{:.4f}".format(0.1))
```

```
0.1000
```

```
In [146]: 1 total, count = (0,) * 2
2 data = [10.5, 11.2, 9.8, None, 11.5, None]
3 for e in data:
```

```
4         if e:
5             total += e
6             count += 1
7     average = total / count
8     print(average)
```

10.75

```
In [147]: 1 for i, e in enumerate(data):
2         if not e:
3             data[i] = average
4         print(data)
```

[10.5, 11.2, 9.8, 10.75, 11.5, 10.75]

```
In [148]: 1 data = [10.5, 11.2, 9.8, None, 11.5, None]
2     count = sum(1 for e in data if e)
3     total = sum(e for e in data if e)
4     average = total / count
5     print(sum, average)
```

<built-in function sum> 10.75

```
In [149]: 1 data = [10.5, 11.2, 9.8, None, 11.5, None]
2     average = statistics.fmean(e for e in data if e)
3     data = [average if e is None else e for e in data]
4     print(data)
```

[10.5, 11.2, 9.8, 10.75, 11.5, 10.75]

```
In [150]: 1 data = [10.5, 11.2, 9.8, None, 11.5, None]
2     df = pandas.DataFrame(data)
3     df.fillna(df.mean(), inplace = True)
4     print(df[0].tolist())
```

[10.5, 11.2, 9.8, 10.75, 11.5, 10.75]

```
In [151]: 1 print(len(str((1, 2))))
```

6

```
In [152]: 1 print(str((1, 2)))
```

(1, 2)

```
In [153]: 1 m, n = 4, 5
2     for i in range(1, m):
3         for j in range(1, n):
4             print(f"{i} x {j} = {i * j}")
5     print("-" * 8)
```

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
-----
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
-----
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
-----
```

In [154]:

```
1
2
3 data = ("2021-01-01", 10, 20], ["2021-01-02", 20, 18], ["2021-01-03", -10,
4 10], ["2021-01-04", 100, 102], ["2021-01-05", 20, 45])
5 largest = 0
6 for e in data:
7     distance = abs(e[2] - e[1])
8     e.append(distance)
9     if distance > largest:
10         dateOfLargest = e[0]
11         largest = distance
12 print(data)
13 print(dateOfLargest)
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
```

```
In [156]: 1 tuple1 = ([0],)
          2 print(type(tuple1))
          3 print(tuple1)
          4 print(id(tuple1))
          5 print(id(tuple1[0]))
          6 tuple1[0][0] += 1
          7 print(tuple1)
          8 print(id(tuple1))
          9 print(id(tuple1[0]))
```

```
<class 'tuple'>
([0],)
128573173295168
128573172611328
([1],)
128573173295168
128573172611328
```

```
In [157]: 1 print(hash(True))
          2 a = True
          3 print(hash(a))
          4 b = True
          5 print(hash(b))
          6 a = b
          7 print(hash(a))
```

```
1
1
1
1
```

```
In [158]: 1 print(hash(257))
          2 a = 257
          3 print(hash(a))
          4 b = 257
          5 print(hash(b))
          6 a = b
          7 print(hash(a))
```

```
257
257
257
257
```

```
In [159]: 1 print(hash(2570))
          2 a = 2570
          3 print(hash(a))
          4 b = 2570
          5 print(hash(b))
          6 a = b
          7 print(hash(a))
```

```
2570
2570
2570
2570
```

```
In [160]: 1 print(hash(25))
          2 a = 25
          3 print(hash(a))
          4 b = 25
          5 print(hash(b))
          6 a = b
          7 print(hash(a))
```

```
25
25
25
25
```

```
In [161]: 1 print(hash(2.5))
          2 a = 2.5
          3 print(hash(a))
          4 b = 2.5
          5 print(hash(b))
          6 print(hex(hash(b)))
          7 a = b
          8 print(len(hex(hash(a))))
          9 print(hash(b))
```

```
1152921504606846978
1152921504606846978
1152921504606846978
0x10000000000000002
18
1152921504606846978
```

```
In [162]: 1 for i in range(5):
          2     for j in range(4):
          3         pass
          4 print(i, j)
```

```
4 3
```

```
In [163]: 1 s1 = """
          2 "'And' and 'or' are the basic operations of logic. Together with 'no' (the
          3 logical operation
          4 of negation) they are a complete set of basic logical operations – all other
          5 logical
          6 operations, no matter how complex, can be obtained by suitable combinations of
          7 these."
          8 — John von Neumann, The Computer and the Brain
          9 """
          10
          11 s2 = """
          12 Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
          13 incididunt ut
```

```

7 labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud
exercitation ullamco laboris
8 nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in
reprehenderit in voluptate velit
9 esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat
1 non proident, sunt
0 in culpa qui officia deserunt mollit anim id est laborum.
1 """
1
1 charCounts = {}
2 for c in s1:
1     if (ord(c) >= ord("A") and ord(c) <= ord("Z")) or (ord(c) >= ord("a")
3 and ord(c) <= ord("z")):
1         charCounts[c] = charCounts.get(c, 0) + 1
4 print(charCounts)
1
5
1
6
1
7
1
8
1
9
2
0

```

```

{'A': 1, 'n': 20, 'd': 4, 'a': 23, 'o': 31, 'r': 12, 'e': 27, 't': 21, 'h': 11, 'b': 7, 's': 9, 'i':
17, 'c': 10, 'p': 7, 'f': 4, 'l': 12, 'g': 6, 'T': 2, 'w': 2, 'y': 2, 'm': 6, 'x': 1, 'u': 3, 'J':
1, 'v': 1, 'N': 1, 'C': 1, 'B': 1}

```

```

In [164]: 1 d1 = {"a": 10, "b": 20, "c": 30}
2 d2 = {"d": 100, "e": 200, "f": 300}
3 d3 = {"f": 30, "g": 40}
4 keysList, valuesList = [], []
5 for d in d1, d2, d3:
6     for k, v in d.items():
7         keysList.append(k)
8         valuesList.append(v)
9 print(keysList)
10 print(valuesList)

```

```

['a', 'b', 'c', 'd', 'e', 'f', 'f', 'g']
[10, 20, 30, 100, 200, 300, 30, 40]

```

```

In [165]: 1 keysList, valuesList = [], []
2 keysList = list(d1.keys())
3 keysList.extend(list(d2.keys()))
4 keysList.extend(list(d3.keys()))
5 print(keysList)

```

```

['a', 'b', 'c', 'd', 'e', 'f', 'f', 'g']

```

```
In [166]: 1 valuesList = list(d1.values())
          2 valuesList.extend(list(d2.values()))
          3 valuesList.extend(list(d3.values()))
          4 print(valuesList)
```

```
[10, 20, 30, 100, 200, 300, 30, 40]
```

```
In [167]: 1 print(list(d1.keys()) + list(d2.keys()) + list(d3.keys()))
          2 print(list(d1.values()) + list(d2.values()) + list(d3.values()))
```

```
['a', 'b', 'c', 'd', 'e', 'f', 'f', 'g']
[10, 20, 30, 100, 200, 300, 30, 40]
```

```
In [168]: 1 grades = {"John": [90, 95, 98], "Eric": [86, 84, 92], "Michael": [90, 89, 85]}
          2 exam = {"Eric": 99, "John": 100}
          3 for k, v in grades.items():
          4     if k in exam:
          5         v.insert(0, exam[k])
          6     else:
          7         v.insert(0, None)
          8 print(grades)
```

```
{'John': [100, 90, 95, 98], 'Eric': [99, 86, 84, 92], 'Michael': [None, 90, 89, 85]}
```

```
In [169]: 1 for k, v in grades.items():
          2     score = exam.get(k)
          3     grades[k] = [score] + grades[k]
          4 print(grades)
```

```
{'John': [100, 100, 90, 95, 98], 'Eric': [99, 99, 86, 84, 92], 'Michael': [None, None, 90, 89, 85]}
```

```
In [170]: 1 for k, v in grades.items():
          2     v.insert(0, exam.get(k))
          3 print(grades)
```

```
{'John': [100, 100, 100, 90, 95, 98], 'Eric': [99, 99, 99, 86, 84, 92], 'Michael': [None, None, None, 90, 89, 85]}
```

```
In [171]: 1 for k in grades:
          2     print(k)
```

```
John
Eric
Michael
```

```
In [172]: 1 l = ["AAPL", "AAPL", "Aapl", "aapl", "MSFT"]
          2 print(list(set(l)))
```

```
['AAPL', 'Aapl', 'MSFT', 'aapl']
```

```
In [173]: 1 l = [e.casefold() for e in l]
          2 print(list(set(l)))
```

```
['aapl', 'msft']
```

```
In [174]: 1 se = {e.casefold() for e in l}
          2 print(se)
```

```
{'aapl', 'msft'}
```

```
In [175]: 1 data = {"d1": {"a": 1, "b": 2, "c": 3}, "d2": {"b": 20, "c": 30, "d": 40},
          2 "d3": {"d": 100, "x": 200}}
          3 uniqueKeys = {k for v in (data["d1"], data["d2"], data["d3"]) for k in v}
          print(uniqueKeys)
```

```
{'d', 'x', 'c', 'b', 'a'}
```

```
In [176]: 1 try:
          2     s1 = {[0, 1]}
          3     print(s1)
          4 except TypeError as ex:
          5     print(type(ex)., ":", " ", *ex.args, ".", sep = " ")
```

```
TypeError: cannot use 'list' as a set element (unhashable type: 'list').
```

```
In [177]: 1 m = [0] * 3
          2 print(m)
          3 print(m[0] is m[1])
```

```
[0, 0, 0]
True
```

```
In [178]: 1 m = [[0] * 3 for _ in range(3)]
          2 print(m)
          3 print(m[0] is m[1])
          4 print(m[0][0] is m[0][1])
```

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
False
True
```

```
In [179]: 1 m = [[10000 for i in range(3)] for _ in range(4)]
          2 print(m)
```

```
[[10000, 10000, 10000], [10000, 10000, 10000], [10000, 10000, 10000], [10000, 10000, 10000]]
```

```
In [180]: 1 pprint.pprint(m, width = 1)
```

```
[[10000,
 10000,
 10000],
 [10000,
 10000,
 10000],
 [10000,
 10000,
 10000],
 [10000,
 10000,
 10000]]
```

```
In [181]: 1 print(m[0] is m[1])
          2 print(m[0][0] is m[0][1])
          3 print(hash(m[0][0]), hash(m[0][1]))
```

```
False
True
10000 10000
```

```
In [182]: 1 print(id(m[0][0]), id(m[0][1]))
```

```
128573172653872 128573172653872
```

```
In [183]: 1 m[0][0] = 1001
          2 pprint.pprint(m)
```

```
[[1001, 10000, 10000],
 [10000, 10000, 10000],
 [10000, 10000, 10000],
 [10000, 10000, 10000]]
```

```
In [184]: 1 print(id(m[0][0]), id(m[0][1]))
          2 print(m[0][0] is m[0][1])
```

```
128573172653712 128573172653872
False
```

```
In [185]: data = [{"open": 100, "high": 120, "low": 90, "close": 110}, {"open": 110,
1 "high": 130, "low": 80, "close": 120}, {"open": 120, "high": 140, "low": 70,
2 "close": 130}, {"open": 130, "high": 150, "low": 60, "close": 140},]
3 ranges = []
4 for d in data:
5     ranges.append(d["high"] - d["low"])
print(ranges)
```

```
[30, 50, 70, 90]
```

```
In [186]: data = [{"open": 100, "high": 120, "low": 90, "close": 110}, {"open": 110,
1 "high": 130, "low": 80, "close": 120}, {"open": 120, "high": 140, "low": 70,
2 "close": 130}, {"open": 130, "high": 150, "low": 60, "close": 140},]
3 ranges = [d["high"] - d["low"] for d in data]
print(ranges)
```

```
[30, 50, 70, 90]
```

```
In [187]: 1 bogusComposites = {i for i in range(1, 101) for j in range(2, 10) if i % j ==
          2 0}
          3 allNumbers = {i for i in range(11, 101)} | {1}
          print(allNumbers - bogusComposites)
```

```
{1, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97}
```

```
In [188]: 1 allNumbers = set(range(1, 101))
          2 print(allNumbers - bogusComposites)
```

```
{1, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97}
```

In [189]:

```
data = [{"symbol": "ABCD", "name": "ABCD Company", "ranking": 2, "risk": 0.2},
{"symbol": "BCDE", "name": "BCDE Company", "ranking": 5, "risk": 0.2},
{"symbol": "CDEF", "name": "CDEF Company", "ranking": 8, "risk": 0.5},
{"symbol": "DEFG", "name": "DEFG Company", "ranking": 7, "risk": 0.8},
{"symbol": "EFGH", "name": "EFGH Company", "ranking": 9, "risk": 0.6},
{"symbol": "FGHI", "name": "FGHI Company", "ranking": 10, "risk": 0.1},
{"symbol": "GHIJ", "name": "GHIJ Company", "ranking": 3, "risk": 0.6},
{"symbol": "HIJK", "name": "HIJK Company", "ranking": 5, "risk": 0.5},
{"symbol": "IJKL", "name": "IJKL Company", "ranking": 5, "risk": 0.7},
{"symbol": "JKLM", "name": "JKLM Company", "ranking": 6, "risk": 0.9},
{"symbol": "KLMN", "name": "KLMN Company", "ranking": 6, "risk": 0.4},
{"symbol": "LMNO", "name": "LMNO Company", "ranking": 8, "risk": 0.4},
{"symbol": "MNOP", "name": "MNOP Company", "ranking": 8, "risk": 0.2},
{"symbol": "NOPQ", "name": "NOPQ Company", "ranking": 1, "risk": 0.5},
{"symbol": "OPQR", "name": "OPQR Company", "ranking": 9, "risk": 0.2},
{"symbol": "PQRS", "name": "PQRS Company", "ranking": 10, "risk": 0.9},
{"symbol": "QRST", "name": "QRST Company", "ranking": 3, "risk": 0.4},
{"symbol": "RSTU", "name": "RSTU Company", "ranking": 7, "risk": 0.3},
{"symbol": "STUV", "name": "STUV Company", "ranking": 8, "risk": 0.1},
{"symbol": "TUVW", "name": "TUVW Company", "ranking": 7, "risk": 0.9}]
filtered = [d for d in data if d["ranking"] > 5 and d["risk"] < 0.6]
print(filtered)
```

```
[{'symbol': 'CDEF', 'name': 'CDEF Company', 'ranking': 8, 'risk': 0.5}, {'symbol': 'FGHI', 'name': 'FGHI Company', 'ranking': 10, 'risk': 0.1}, {'symbol': 'KLMN', 'name': 'KLMN Company', 'ranking': 6, 'risk': 0.4}, {'symbol': 'LMNO', 'name': 'LMNO Company', 'ranking': 8, 'risk': 0.4}, {'symbol': 'MNOP', 'name': 'MNOP Company', 'ranking': 8, 'risk': 0.2}, {'symbol': 'OPQR', 'name': 'OPQR Company', 'ranking': 9, 'risk': 0.2}, {'symbol': 'RSTU', 'name': 'RSTU Company', 'ranking': 7, 'risk': 0.3}, {'symbol': 'STUV', 'name': 'STUV Company', 'ranking': 8, 'risk': 0.1}]
```

In [190]:

```
1 for d in filtered:
2     d["weight"] = float(f"{d["ranking"] * d["risk"]:.2f}")
3 print(filtered)
```

```
[{'symbol': 'CDEF', 'name': 'CDEF Company', 'ranking': 8, 'risk': 0.5, 'weight': 4.0}, {'symbol': 'FGHI', 'name': 'FGHI Company', 'ranking': 10, 'risk': 0.1, 'weight': 1.0}, {'symbol': 'KLMN', 'name': 'KLMN Company', 'ranking': 6, 'risk': 0.4, 'weight': 2.4}, {'symbol': 'LMNO', 'name': 'LMNO Company', 'ranking': 8, 'risk': 0.4, 'weight': 3.2}, {'symbol': 'MNOP', 'name': 'MNOP Company', 'ranking': 8, 'risk': 0.2, 'weight': 1.6}, {'symbol': 'OPQR', 'name': 'OPQR Company', 'ranking': 9, 'risk': 0.2, 'weight': 1.8}, {'symbol': 'RSTU', 'name': 'RSTU Company', 'ranking': 7, 'risk': 0.3, 'weight': 2.1}, {'symbol': 'STUV', 'name': 'STUV Company', 'ranking': 8, 'risk': 0.1, 'weight': 0.8}]
```

In [191]:

```
1 weightedStocks = [dict(symbol = d["symbol"], weight = d["weight"]) for d in
2 filtered]
3 print(weightedStocks)
```

```
[{'symbol': 'CDEF', 'weight': 4.0}, {'symbol': 'FGHI', 'weight': 1.0}, {'symbol': 'KLMN', 'weight': 2.4}, {'symbol': 'LMNO', 'weight': 3.2}, {'symbol': 'MNOP', 'weight': 1.6}, {'symbol': 'OPQR', 'weight': 1.8}, {'symbol': 'RSTU', 'weight': 2.1}, {'symbol': 'STUV', 'weight': 0.8}]
```

In [192]:

```
1 weightedStocks = [dict(symbol = d["symbol"], weight = round(d["ranking"] *
2 d["risk"], 2)) for d in data if d["ranking"] >= 5 and d["risk"] < 0.6]
3 print(weightedStocks)
```

```
[{'symbol': 'BCDE', 'weight': 1.0}, {'symbol': 'CDEF', 'weight': 4.0}, {'symbol': 'FGHI', 'weight': 1.0}, {'symbol': 'HIJK', 'weight': 2.5}, {'symbol': 'KLMN', 'weight': 2.4}, {'symbol': 'LMNO', 'weight': 3.2}, {'symbol': 'MNOP', 'weight': 1.6}, {'symbol': 'OPQR', 'weight': 1.8}, {'symbol': 'RSTU', 'weight': 2.1}, {'symbol': 'STUV', 'weight': 0.8}]
```

```
In [193]: 1 weightedStocks = [{"symbol": d["symbol"], "weight": round(d["ranking"] *  
2 d["risk"], 2)} for d in data if d["ranking"] >= 5 and d["risk"] < 0.6]  
print(weightedStocks)
```

```
[{'symbol': 'BCDE', 'weight': 1.0}, {'symbol': 'CDEF', 'weight': 4.0}, {'symbol': 'FGHI', 'weight': 1.0}, {'symbol': 'HIJK', 'weight': 2.5}, {'symbol': 'KLMN', 'weight': 2.4}, {'symbol': 'LMNO', 'weight': 3.2}, {'symbol': 'MNOP', 'weight': 1.6}, {'symbol': 'OPQR', 'weight': 1.8}, {'symbol': 'RSTU', 'weight': 2.1}, {'symbol': 'STUV', 'weight': 0.8}]
```

```
In [194]: 1 m = [[0] * 2] * 3  
2 print(m)  
3 for r in range(len(m)):  
4     for c in range(len(m[r])):  
5         print(id(m[r][c]))
```

```
[[0, 0], [0, 0], [0, 0]]  
11370064  
11370064  
11370064  
11370064  
11370064  
11370064
```

```
In [195]: 1 m = [[{0}] * 2] * 3  
2 print(m)  
3 for r in range(len(m)):  
4     for c in range(len(m[r])):  
5         print(id(m[r][c]))
```

```
[[{0}, {0}], [{0}, {0}], [{0}, {0}]]  
128573172485248  
128573172485248  
128573172485248  
128573172485248  
128573172485248  
128573172485248
```

```
In [196]: 1 m[0][0] = 1  
2 for r in range(len(m)):  
3     for c in range(len(m[r])):  
4         print(id(m[r][c]))  
5 print(m)
```

```
11370096  
128573172485248  
11370096  
128573172485248  
11370096  
128573172485248  
[[1, {0}], [1, {0}], [1, {0}]]
```

```
In [197]: 1 m = [[{0} for _ in range(2)] for _ in range(3)]
          2 print(m)
          3 for r in range(len(m)):
          4     for c in range(len(m[r])):
          5         print(id(m[r][c]))
          6 m[0][0] = 1
          7 print(m)
```

```
[[{0}, {0}], [{0}, {0}], [{0}, {0}]]
128573172485696
128573172482336
128573172482112
128573172486144
128573172487040
128573172485024
[[1, {0}], [{0}, {0}], [{0}, {0}]]
```

```
In [198]: 1 m = [[{0} for _ in range(2)] for _ in range(3)]
          2 print(m)
          3 for r in range(len(m)):
          4     for c in range(len(m[r])):
          5         print(id(m[r][c]))
          6 m[0][0] = {1}
          7 print(m)
```

```
[[{0}, {0}], [{0}, {0}], [{0}, {0}]]
128573172485696
128573172483680
128573172487488
128573172486816
128573172486592
128573172488160
[[{1}, {0}], [{0}, {0}], [{0}, {0}]]
```

```
In [199]: 1 m = [[0 for _ in range(2)] for _ in range(3)]
          2 for r in range(len(m)):
          3     for c in range(len(m[r])):
          4         print(id(m[r][c]))
```

```
11370064
11370064
11370064
11370064
11370064
11370064
```

```
In [200]: 1 m = [[10000 for _ in range(2)] for _ in range(3)]
          2 for r in range(len(m)):
          3     for c in range(len(m[r])):
          4         print(id(m[r][c]))
          5 m[0][0] = 1
          6 print(m)
```

```
128573172653264
128573172653264
128573172653264
128573172653264
128573172653264
128573172653264
[[1, 10000], [10000, 10000], [10000, 10000]]
```

```
In [201]: 1 m = [{0} for _ in range(2)] for _ in range(3)
2         for r in range(len(m)):
3             for c in range(len(m[r])):
4                 print(id(m[r][c]))
```

```
128573172483680
128573172487488
128573172485248
128573172486144
128573172488160
128573172486368
```

```
In [202]: 1 # print(input("Text box displayed ?"))
```

```
In [203]: 1 a = 0
2         b = complex(real = 3.0, imag = 4)
3         print(repr(a))
4         print(repr(b))
5         print(b)
```

```
0
(3+4j)
(3+4j)
```

```
In [204]: 1 values = [4, 3, 1, 2]
2         try:
3             minimum = abs(values[0])
4             for value in values[1:]:
5                 if abs(value) < minimum:
6                     minimum = abs(value)
7         except IndexError:
8             minimum = 0
9             pass
10        finally:
11            print(f"Minimum is: {minimum}")
```

```
Minimum is: 1
```

```
In [205]: 1 values = []
2
3         try:
4             minimum = abs(values[0])
5             for value in values[1:]:
6                 if abs(value) < minimum:
7                     minimum = abs(value)
8         except IndexError:
9             minimum = 0
```

```
10     pass
11 finally:
12     print(f"Minimum is: {minimum}")
```

Minimum is: 0

```
In [206]: 1 try:
2         raise ValueError("Custom message")
3 except ValueError as ex:
4     print(type(ex).      + ": " + ex.args[0] + ".")
```

ValueError: Custom message.

```
In [ ]: 1 l = [4, 3, 2, 1, 0]
2 itr = iter(l)
3 try:
4     while True:
5         print(next(itr))
6 except StopIteration as ex:
7     print(type(ex).      , ": ", *ex.args, ".", sep = "")
```

```
4
3
2
1
0
StopIteration: .
```

```
In [208]: 1 try:
2         raise IndexError
3 except IndexError as ex:
4     print(type(ex).      , ": ", *ex.args, ".", sep = "")
```

IndexError: .

```
In [209]: 1 rangeObject = range(10)
2 rangeIterator = iter(rangeObject)
3 print(next(rangeIterator))
4 print(next(rangeIterator))
5 print(next(rangeIterator))
6 print(list(rangeIterator))
```

```
0
1
2
[3, 4, 5, 6, 7, 8, 9]
```

```
In [210]: 1 rangeObj = range(5)
2 print(list(rangeObj))
3 print(list(rangeObj))
```

```
[0, 1, 2, 3, 4]
[0, 1, 2, 3, 4]
```

```
In [211]: 1 enumIterator = enumerate(string.ascii_lowercase)
2 print(enumIterator)
```

```

3 l = list(enumIterator)
4 print(l)
5 l = list(enumIterator)
6 print(dir(type(iter([0]))))
7 print(l)
8 try:
9     print(next(enumIterator))
10 except StopIteration as ex:
11     print(type(ex).      , ": ", *ex.args, ".", sep = "")

```

```

<enumerate object at 0x74efc6458680>
[(0, 'a'), (1, 'b'), (2, 'c'), (3, 'd'), (4, 'e'), (5, 'f'), (6, 'g'), (7, 'h'), (8, 'i'), (9, 'j'),
(10, 'k'), (11, 'l'), (12, 'm'), (13, 'n'), (14, 'o'), (15, 'p'), (16, 'q'), (17, 'r'), (18, 's'),
(19, 't'), (20, 'u'), (21, 'v'), (22, 'w'), (23, 'x'), (24, 'y'), (25, 'z')]
['_class_', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
'__getattr__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__',
'__iter__', '__le__', '__length_hint__', '__lt__', '__ne__', '__new__', '__next__', '__reduce__',
'__reduce_ex__', '__repr__', '__setattr__', '__setstate__', '__sizeof__', '__str__',
'__subclasshook__']
[]
StopIteration: .

```

```

In [212]: 1 print(type(enumIterator))
          2 print(type(iter(enumIterator)))
          3 print(issubclass(type(iter([0])), type(enumIterator)))

```

```

<class 'enumerate'>
<class 'enumerate'>
False

```

```

In [213]: 1 print(iter(enumIterator) is enumIterator)

True

```

```

In [214]: 1 print(type(iter({0, 1, 2})))

<class 'set_iterator'>

```

```

In [215]: 1 print(type(iter({0: "0", 1: "1", "2": 2})))

<class 'dict_keyiterator'>

```

```

In [ ]: 1 l = [4, 3, 2, 1, 0]
        2 itr = iter(l)
        3 for i in itr:
        4     print(i)

```

```

4
3
2
1
0

```

```

In [217]: 1 try:
          2     print(next(l))
          3 except TypeError as ex:
          4     print(type(ex).      , ": ", *ex.args, ".", sep = "")

```

TypeError: 'list' object is not an iterator.

```
In [218]: 1 start = time.perf_counter()
          2 squaresGenerator = (i ** 10 for i in range(1_000_000))
          3 end = time.perf_counter()
          4 print(f"Elapsed: {end - start}")
```

Elapsed: 0.00016390199999705146

```
In [219]: 1 start = time.perf_counter()
          2 squaresListComp = [i ** 10 for i in range(1_000_000)]
          3 end = time.perf_counter()
          4 print(f"Elapsed: {end - start}")
```

Elapsed: 0.23438802800001213

```
In [220]: 1 print((7.16e-5 / 1.824e-1) ** -1)
          2 print(105 / 9)
          3 print(10.5e1 / 9e0)
          4 print(99 / 1)
          5 print(9.9e1 / 1e0) # 1 order of magnitude difference means a multiple of
                                between 10 and 100
```

2547.486033519553
11.666666666666666
11.666666666666666
99.0
99.0

```
In [221]: 1 print(999_999 ** 10)
```

999990000004499988000020999974800020999988000000449999900000001

```
In [222]: 1 print(squaresGenerator is iter(squaresGenerator))
```

True

```
In [223]: 1 l = [10, "abc", 3.14, True]
          2 for i, e in enumerate(l):
          3     print(f"{i}: {e}")
```

0: 10
1: abc
2: 3.14
3: True

```
In [224]: 1 def f(a, *b, x, **y):
          2     print("Positional/Named:", a)
          3     print("Variable positional:", b)
          4     print("Named only:", x)
          5     print("Variable named:", y)
          6
          7
          8 f(0, 1, 2, x = 3, y = 4, z = 5)
```

Positional/Named: 0
Variable positional: (1, 2)
Named only: 3
Variable named: {'y': 4, 'z': 5}

```
In [225]: 1 # f(10, 2, x = 10, 20)
```

```
In [226]: 1 f(10, 2, x = 10, y = 20)
```

Positional/Named: 10
Variable positional: (2,)
Named only: 10
Variable named: {'y': 20}

```
In [227]: 1 f(10, z = 2, x = 10, y = 20)
```

Positional/Named: 10
Variable positional: ()
Named only: 10
Variable named: {'z': 2, 'y': 20}

```
In [228]: 1 f(10, x = 2, y = 10, z = 20)
```

Positional/Named: 10
Variable positional: ()
Named only: 2
Variable named: {'y': 10, 'z': 20}

```
In [229]: 1 # f(a = 10, 2, x = 10, y = 20)
```

```
In [230]: 1 f(x = 10, y = 20, z = 30, a = 10)
```

Positional/Named: 10
Variable positional: ()
Named only: 10
Variable named: {'y': 20, 'z': 30}

```
In [231]: 1 try:
2         f(10, a = 2, x = 10, y = 20)
3     except TypeError as ex:
4         print(type(ex).      )
```

TypeError

```
In [232]: 1 def f1(a, *, x, **y):
2         print(f"a: {a}, x: {x}, y: {y}")
```

```
In [233]: 1 try:
2         f1(10, a = 20, x = 2, y = 10)
3     except TypeError as ex:
4         print(type(ex).      )
```

TypeError

```
In [234]: 1 f1(a = 1, x = 2)
```

```
a: 1, x: 2, y: {}
```

```
In [235]: 1 print(max("cba"))
```

```
c
```

```
In [236]: 1 def f2(a, b):  
2         print(f"a: {a}; b: {b}")  
3  
4  
5 f2(2, 3)
```

```
a: 2; b: 3
```

```
In [237]: 1 f2(2, b = 3)
```

```
a: 2; b: 3
```

```
In [238]: 1 # f2(a = 2, 3)
```

```
In [239]: 1 f2(b = 1, a = 0)
```

```
a: 0; b: 1
```

```
In [240]: 1 # def f3(a = 0, b):  
2         # print(f"a: {a}; b: {b}")
```

```
In [241]: 1 def f3(a, b = 0):  
2         print(f"a: {a}; b: {b}")  
3  
4  
5 f3(-1, 1)
```

```
a: -1; b: 1
```

```
In [242]: 1 # f3(a = -1, 1)
```

```
In [243]: 1 f3(a = -1, b = 1)
```

```
a: -1; b: 1
```

```
In [244]: 1 def f4(a, b = 1, *, x):  
2         print(a, b, x)  
3  
4  
5 f4(0, 2, x = 3)
```

```
0 2 3
```

```
In [245]: 1 def f5(a, b = 1, *, x = 2, y):  
2         print(a, b, x, y)  
3  
4  
5 f5(0, 2, x = 3, y = 4)
```

0 2 3 4

```
In [246]: 1 def f6(**x):
          2     print(x)
          3
          4
          5 f6(a = 1, b = 2, c = 3)
```

{'a': 1, 'b': 2, 'c': 3}

```
In [247]: 1 # def f7(*, **x):
          2 #     print(x)
          3 #
          4 #
          5 # f7(a = 1, b = 2)
```

```
In [248]: 1 def f8(a, *, b):
          2     print(f"a: {a}; b: {b}")
          3
          4
          5 f8(b = 1, a = 0)
```

a: 0; b: 1

```
In [249]: 1 def toJSON(a, *, b, **c):
          2     print(a, b, c)
          3
          4
          5 toJSON(0, b = 3, c1 = 0, c2 = 1)
```

0 3 {'c1': 0, 'c2': 1}

```
In [250]: 1 def avg(a, *b):
          2     return (sum(b) + a) / (len(b) + 1)
          3
          4
          5 print(avg(0, 1, 2, 3, 4))
          6 print(avg(0))
          7 print(avg(0, 1))
```

2.0
0.0
0.5

```
In [251]: 1 def separator(b = 10, *, a = "-"):
          2     return a * b
          3
          4
          5 print(separator())
          6 print(separator(a = "A"))
          7 print(separator(5))
          8 print(separator(a = "B", b = 5))
          9 print(separator(5, a = "B"))
```

```
-----  
AAAAAAAAA  
-----  
BBBBBB  
BBBBBB
```

```
In [252]: 1 print(sep = "|", *("AB", "CD"))
```

```
AB|CD
```

```
In [253]: 1 # def f(a, *b, c):  
2         #     print(a, b, c)  
3         #  
4         #  
5         # f(c = 2, 0, 1, 2)
```

```
In [254]: 1 # def f(*a, b = 0):  
2         #     print(a, b)  
3         #  
4         #  
5         # f(b = 1, 0, 2)
```

```
In [255]: 1 try:  
2         f(b = 1, *(0, 2))  
3 except TypeError as ex:  
4         print(type(ex).      )
```

```
TypeError
```

```
In [256]: 1 # def f(a, *, b):  
2         #     print(a, b)  
3         #  
4         #  
5         # f(b = 1, 0)
```

```
In [257]: 1 # def f(a, b):  
2         #     print(a, b)  
3         #  
4         #  
5         # f(b = 1, 0)
```

```
In [258]: 1 # def f(a, b = 2):  
2         #     print(a, b)  
3         #  
4         #  
5         # f(b = 1, 0)
```

```
In [259]: 1 def f(a, b = 2):  
2         print(a, b)  
3
```

```
4
5 f(b = 1, *(0,))
```

```
0 1
```

```
In [260]: 1 anonymous = lambda iterable: len(set(iterable))
2          print(anonymous("abcd"))
3          print(anonymous("abab"))
4          try:
5              print(anonymous())
6          except TypeError as ex:
7              print(type(ex).      )
8          print(anonymous(""))
9          print(anonymous([0, 1, 2]))
10         print(anonymous([0]))
11         print(anonymous([]))
12         print(anonymous([1, 1, 2, 2, 3, 4]))
```

```
4
2
TypeError
0
3
1
0
4
```

```
In [261]: 1 s = "This is the first sentence. This is the scecond sentence. This is not the
2          fourth sentence, it is the third sentence."
3
4          def f(s = ""):
5              return dict(collections.Counter(s))
6
7
8          print(f(s))
9          print(f(""))
```

```
{'T': 3, 'h': 9, 'i': 10, 's': 13, ' ': 20, 't': 13, 'e': 17, 'f': 2, 'r': 3, 'n': 10, 'c': 6, 'l': 3, 'o': 3, 'd': 2, 'u': 1, ',': 1}
{}
```

```
In [262]: 1 s = "This is the first sentence. This is the scecond sentence. This is not the
2          fourth sentence, it is the third sentence."
3
4          def f(s = ""):
5              words = re.split("\\.| |", s)
6              for word in words:
7                  if word == "":
8                      words.remove(word)
9              frequencies = dict.fromkeys(set(words), 0)
10             for word in words:
11                 frequencies[word] += 1
12             print(frequencies)
```

```
7
8 f(s)
9
10
11
12
13
14
15
```

```
{'is': 4, 'the': 4, 'fourth': 1, 'third': 1, 'it': 1, 'not': 1, 'first': 1, 'second': 1, 'sentence': 4, 'This': 3}
```

```
In [263]: 1 def f(s = ""):
2         s = s.replace(".", "")
3         s = s.replace(",", "")
4         words = s.split()
5         return dict(collections.Counter(words))
6
7
8 print(f(s))
```

```
{'This': 3, 'is': 4, 'the': 4, 'first': 1, 'sentence': 4, 'second': 1, 'not': 1, 'fourth': 1, 'it': 1, 'third': 1}
```

```
In [264]: 1 print(round(1.812))
```

```
2
```

```
In [265]: 1 n, d = (0.3).as_integer_ratio()
2         print(n, d)
3         print(1 / 3)
```

```
5404319552844595 18014398509481984
0.3333333333333333
```

```
In [266]: 1 print(round(0.325, 2))
```

```
0.33
```

```
In [267]: 1 print(f"{0.325:.40f}")
```

```
0.3250000000000000111022302462515654042363
```

```
In [268]: 1 def toJSON(a, *, b, **c):
2         print(a, b, c)
3
```

```

4
5 try:
6     toJSONN(0, 1, 2, b = 3, c1 = 4, c2 = 5)
7 except TypeError as ex:
8     print(type(ex).

```

TypeError

```

In [269]: 1 def toJSONN(a, *, b, **c):
2         print(a, b, c)
3
4
5 toJSONN(0, b = 3, c1 = 4, c2 = 5)

```

0 3 {'c1': 4, 'c2': 5}

```

In [270]: 1 # def toJSONN(a, *):
2         #     print(a)
3         #
4         #
5         # toJSONN(0, 1, 2)

```

```

In [271]: 1 def toJSONN(a, *args):
2         print(a, args)
3
4
5 toJSONN(0, 1, 2)

```

0 (1, 2)

```

In [272]: 1 l = ["AB CD", "EF CC", "GH BC", "IJ AC", "KL CA", "MN JQ", "OP BA"]
2         print(sorted(l, key = lambda e: (e[3], e[4])))

```

['IJ AC', 'OP BA', 'GH BC', 'KL CA', 'EF CC', 'AB CD', 'MN JQ']

```

In [273]: 1 # print(sorted(l, key = lambda e: e[len(e) - i] for i in range(len(e))))
2         print(sorted(l, key = lambda e: e[len(e) - 1]))

```

['KL CA', 'OP BA', 'EF CC', 'GH BC', 'IJ AC', 'AB CD', 'MN JQ']

```

In [274]: 1 a = (1 for _ in range(5))
2         print(a)
3         print(lambda _: "ABCD")
4         print(sorted)
5         print(range)

```

<generator object <genexpr> at 0x74efc64a24d0>
 <function <lambda> at 0x74efc644b740>
 <built-in function sorted>
 <class 'range'>

```

In [275]: 1 def f(*a, b):
2         print(a, b)
3

```

```
4
5 f(b = 0, *(1, 2))
```

```
(1, 2) 0
```

```
In [276]: 1 def f(a, *b, c):
          2     print(a, b, c)
          3
          4
          5 f(0, c = 1, *(2, 3))
```

```
0 (2, 3) 1
```

```
In [277]: 1 print(statistics.fmean)
```

```
<function fmean at 0x74effaaa6a30>
```

```
In [278]: 1 print(dict([("a", 97), ("b", 98), ("c", 99)]))
```

```
{'a': 97, 'b': 98, 'c': 99}
```

```
In [279]: 1 widgets = [f"w{i}" for i in range(1, 21)]
          2 skus = [f"sku{i}" for i in range(1, len(widgets) + 1)]
          3 print(widgets)
          4 print(skus)
```

```
['w1', 'w2', 'w3', 'w4', 'w5', 'w6', 'w7', 'w8', 'w9', 'w10', 'w11', 'w12', 'w13', 'w14', 'w15',
'w16', 'w17', 'w18', 'w19', 'w20']
['sku1', 'sku2', 'sku3', 'sku4', 'sku5', 'sku6', 'sku7', 'sku8', 'sku9', 'sku10', 'sku11', 'sku12',
'sku13', 'sku14', 'sku15', 'sku16', 'sku17', 'sku18', 'sku19', 'sku20']
```

```
In [280]: 1 print({w: s for w, s in zip(widgets, skus)})
```

```
{'w1': 'sku1', 'w2': 'sku2', 'w3': 'sku3', 'w4': 'sku4', 'w5': 'sku5', 'w6': 'sku6', 'w7': 'sku7',
'w8': 'sku8', 'w9': 'sku9', 'w10': 'sku10', 'w11': 'sku11', 'w12': 'sku12', 'w13': 'sku13', 'w14':
'sku14', 'w15': 'sku15', 'w16': 'sku16', 'w17': 'sku17', 'w18': 'sku18', 'w19': 'sku19', 'w20':
'sku20'}
```

```
In [281]: 1 print(dict(zip(widgets, skus)))
```

```
{'w1': 'sku1', 'w2': 'sku2', 'w3': 'sku3', 'w4': 'sku4', 'w5': 'sku5', 'w6': 'sku6', 'w7': 'sku7',
'w8': 'sku8', 'w9': 'sku9', 'w10': 'sku10', 'w11': 'sku11', 'w12': 'sku12', 'w13': 'sku13', 'w14':
'sku14', 'w15': 'sku15', 'w16': 'sku16', 'w17': 'sku17', 'w18': 'sku18', 'w19': 'sku19', 'w20':
'sku20'}
```

```
In [282]: 1 suits = "shdc" # Spades, Hearts, Diamonds, Clubs
          2 ranks = list("23456789") + ["10", "J", "Q", "K", "A"]
          3 print(suits)
          4 print(ranks)
```

```
shdc
['2', '3', '4', '5', '6', '7', '8', '9', '10', 'J', 'Q', 'K', 'A']
```

```
In [283]: 1 print([(rank, suit) for rank in ranks] for suit in suits))
```

```
[('2', 's'), ('3', 's'), ('4', 's'), ('5', 's'), ('6', 's'), ('7', 's'), ('8', 's'), ('9', 's'), ('10', 's'), ('J', 's'), ('Q', 's'), ('K', 's'), ('A', 's')], [('2', 'h'), ('3', 'h'), ('4', 'h'), ('5', 'h'), ('6', 'h'), ('7', 'h'), ('8', 'h'), ('9', 'h'), ('10', 'h'), ('J', 'h'), ('Q', 'h'), ('K', 'h'), ('A', 'h')], [('2', 'd'), ('3', 'd'), ('4', 'd'), ('5', 'd'), ('6', 'd'), ('7', 'd'), ('8', 'd'), ('9', 'd'), ('10', 'd'), ('J', 'd'), ('Q', 'd'), ('K', 'd'), ('A', 'd')], [('2', 'c'), ('3', 'c'), ('4', 'c'), ('5', 'c'), ('6', 'c'), ('7', 'c'), ('8', 'c'), ('9', 'c'), ('10', 'c'), ('J', 'c'), ('Q', 'c'), ('K', 'c'), ('A', 'c')]]
```

```
In [284]: 1 pprint.pprint([(rank, suit) for rank in ranks] for suit in suits])
```

```
[('2', 's'),
 ('3', 's'),
 ('4', 's'),
 ('5', 's'),
 ('6', 's'),
 ('7', 's'),
 ('8', 's'),
 ('9', 's'),
 ('10', 's'),
 ('J', 's'),
 ('Q', 's'),
 ('K', 's'),
 ('A', 's')],
 [('2', 'h'),
 ('3', 'h'),
 ('4', 'h'),
 ('5', 'h'),
 ('6', 'h'),
 ('7', 'h'),
 ('8', 'h'),
 ('9', 'h'),
 ('10', 'h'),
 ('J', 'h'),
 ('Q', 'h'),
 ('K', 'h'),
 ('A', 'h')],
 [('2', 'd'),
 ('3', 'd'),
 ('4', 'd'),
 ('5', 'd'),
 ('6', 'd'),
 ('7', 'd'),
 ('8', 'd'),
 ('9', 'd'),
 ('10', 'd'),
 ('J', 'd'),
 ('Q', 'd'),
 ('K', 'd'),
 ('A', 'd')],
 [('2', 'c'),
 ('3', 'c'),
 ('4', 'c'),
 ('5', 'c'),
 ('6', 'c'),
 ('7', 'c'),
 ('8', 'c'),
 ('9', 'c'),
 ('10', 'c'),
 ('J', 'c'),
 ('Q', 'c'),
 ('K', 'c'),
 ('A', 'c')]]
```

```
In [285]: 1 print([list(zip(ranks, s * 13)) for s in suits])
```

```
[(('2', 's'), ('3', 's'), ('4', 's'), ('5', 's'), ('6', 's'), ('7', 's'), ('8', 's'), ('9', 's'), ('10', 's'), ('J', 's'), ('Q', 's'), ('K', 's'), ('A', 's'))], [('2', 'h'), ('3', 'h'), ('4', 'h'), ('5', 'h'), ('6', 'h'), ('7', 'h'), ('8', 'h'), ('9', 'h'), ('10', 'h'), ('J', 'h'), ('Q', 'h'), ('K', 'h'), ('A', 'h')], [('2', 'd'), ('3', 'd'), ('4', 'd'), ('5', 'd'), ('6', 'd'), ('7', 'd'), ('8', 'd'), ('9', 'd'), ('10', 'd'), ('J', 'd'), ('Q', 'd'), ('K', 'd'), ('A', 'd')], [('2', 'c'), ('3', 'c'), ('4', 'c'), ('5', 'c'), ('6', 'c'), ('7', 'c'), ('8', 'c'), ('9', 'c'), ('10', 'c'), ('J', 'c'), ('Q', 'c'), ('K', 'c'), ('A', 'c')]]
```

In [286]:

```
1 def f(list1, *, reverse = False):
2     if not reverse:
3         return sorted(list1)
4     else:
5         return sorted(list1, reverse = True)
6
7
8 print(f([3, 2, 4, 5, 1, 6, 8, 7]))
9 print(f([3, 2, 4, 5, 1, 6, 8, 7], reverse = True))
10 print(f([3, 2, 4, 5, 1, 6, 8, 7])[0])
11 print(f([3, 2, 4, 5, 1, 6, 8, 7], reverse = True)[0])
```

```
[1, 2, 3, 4, 5, 6, 7, 8]
[8, 7, 6, 5, 4, 3, 2, 1]
1
8
```

In [287]:

```
1 def f(list1, *, reverse = False):
2     return sorted(list1, reverse = reverse)
3
4
5 print(f([3, 2, 4, 5, 1, 6, 8, 7]))
6 print(f([3, 2, 4, 5, 1, 6, 8, 7], reverse = True))
7 print(f([3, 2, 4, 5, 1, 6, 8, 7])[0])
8 print(f([3, 2, 4, 5, 1, 6, 8, 7], reverse = True)[0])
```

```
[1, 2, 3, 4, 5, 6, 7, 8]
[8, 7, 6, 5, 4, 3, 2, 1]
1
8
```

In [288]:

```
1 print(hex(ord(chr(0xE3).upper())))
```

0xc3

In [289]:

```
1 print("abc" in "defabcghi")
```

True

In [290]:

```
1 print("abc" in "abbde")
```

False

In [291]:

```
1 def solve(a, b, c):
2     solution = ((-1 * b) + math.sqrt((b * b) - (4 * a * c))) / (2 * a)
3     return round(solution, 2)
4
5
```

```
6 print(solve(3, 4, -5))
7 print(solve(1, -5, -8))
```

```
0.79
6.27
```

```
In [292]: 1 def encrypt(s):
          2     return "".join(chr(ord(c) + 10) for c in s)
          3
          4
          5 print(encrypt("ABCDEFGHIJKLMNOPQRSTUVWXYZ"))
```

```
KL MN OP QR ST UV WX YZ [\]^_`abcd
```

```
In [293]: 1 def decrypt(s):
          2     return "".join(chr(ord(c) - 10) for c in s)
          3
          4
          5 print(decrypt("KL MN OP"))
          6 print(decrypt("S}kkm*%o\x81~yx"))
```

```
ABCDEF
Isaac Newton
```

```
In [294]: 1 print(10 % 3)
```

```
1
```

```
In [295]: 1 a = array.array("l", [20])
          2 print(a)
```

```
array('l', [20])
```

```
In [296]: 1 for i in a:
          2     print(i)
```

```
20
```

```
In [297]: 1 print(sys.getsizeof(a))
```

```
88
```

```
In [298]: 1 try:
          2     print(sys.getallocatedblocks(a))
          3 except TypeError as e:
          4     print(type(e).      )
```

```
TypeError
```

```
In [299]: 1 print(sys.getsizeof(1))
```

```
28
```

```
In [300]: 1 def f(s):
          2     print(s)
          3
```

```

4
5 try:
6     f(*"hello")
7 except TypeError as ex:
8     print(type(ex).      , ex.args)

```

TypeError ('f()' takes 1 positional argument but 5 were given',)

```

In [301]: 1 def func(s):
2           print(s)
3
4
5 def apply(func, *str):
6     result = func(*str)
7     return result
8
9
10 apply(func, "john")

```

john

```

In [302]: 1 try:
2           apply(func, "john", "smith")
3 except TypeError as ex:
4     print(type(ex).      , *ex.args)

```

TypeError func() takes 1 positional argument but 2 were given

```

In [303]: 1 def encloseGetPower(b):
2           print("hex(id(b)):", hex(id(b)))
3
4           def getPower(e):
5               print("hex(id(e)):", hex(id(e)))
6
7               return b ** e
8
9           print("getPower:", getPower)
10
11          return getPower
12
13
14 getBinaryPower = encloseGetPower(2)
15 print(getBinaryPower(31))
16 getTertiaryPower = encloseGetPower(3)
17 print(getTertiaryPower(4))

```

```

hex(id(b)): 0xad7e90
getPower: <function encloseGetPower.<locals>.getPower at 0x74efc644bd70>
hex(id(e)): 0xad8230
2147483648
hex(id(b)): 0xad7eb0
getPower: <function encloseGetPower.<locals>.getPower at 0x74efc644be20>
hex(id(e)): 0xad7ed0
81

```

```
In [304]: 1 print("getBinaryPower:", getBinaryPower)
2 print("encloseGetPower:", encloseGetPower)
3 try:
4     print("getPower:", getPower)
5 except NameError as ex:
6     print(type(ex)., ex.args)
```

```
getBinaryPower: <function encloseGetPower.<locals>.getPower at 0x74efc644bd70>
encloseGetPower: <function encloseGetPower at 0x74efc644b5e0>
NameError ("name 'getPower' is not defined",)
```

```
In [305]: 1 print("getBinaryPower:", getBinaryPower.      )
2 print("encloseGetPower:", encloseGetPower.      )
```

```
getBinaryPower: (<cell at 0x74efc24f1510: int object at 0xad7e90>,)
encloseGetPower: None
```

```
In [306]: 1 def outer(a, b):
2         print(hex(id(a)))
3         sumTotal = a + b
4
5         def inner():
6             prod = a * b
7             print(a, b, sumTotal, prod)
8             return "You just called a closure!"
9
10        return inner
11
12
13 f = outer(2, 3)
```

```
0xad7e90
```

```
In [307]: 1 f = outer(2, 3)
```

```
0xad7e90
```

```
In [308]: 1 print(f)
```

```
<function outer.<locals>.inner at 0x74efc644bed0>
```

```
In [309]: 1 print(f.      )
```

```
(<cell at 0x74efc641f430: int object at 0xad7e90>, <cell at 0x74effaaf9db0: int object at 0xad7eb0>,
<cell at 0x74effb46b430: int object at 0xad7ef0>)
```

```
In [310]: 1 f()
```

```
2 3 5 6
```

```
Out[310]: 'You just called a closure!'
```

```
In [311]: 1 def execute(func):
2
3         def inner(a, b):
```

```

4         return func(a, b)
5
6     return inner
7
8
9 def add(a, b):
10     return a + b
11
12
13 add_executor = execute(add)
14 print(add_executor.

```

(<cell at 0x74efc24f0a30: function object at 0x74efc24f8040>,)

In [312]: 1 print(hex(id(add)))

0x74efc24f8040

In [313]: 1 try:
2 print(hex(add))
3 except TypeError as ex:
4 print(type(ex).)

TypeError

In [314]: 1 add_executor(2, 3)

Out[314]: 5

In [315]: 1 def f():
2 pass
3
4
5 print(f.)

None

In [316]: 1 def f():
2
3 def g():
4 return
5
6 return g
7
8
9 h = f()
10 print(f.)
11 print(h.)

None

None

In [317]: 1 a = 0
2 try:

```
3     print(a.          )
4 except AttributeError as e:
5     print(type(e).    )
```

AttributeError

```
In [318]: 1 a = []
          2 try:
          3     print(a.          )
          4 except AttributeError as e:
          5     print(type(e).    )
```

AttributeError

```
In [319]: 1 def execute(func):
          2
          3     def inner(*posargs, **kwargs):
          4         return func(*posargs, **kwargs)
          5
          6     return inner
          7
          8
          9 def add(a, b, c):
         10     return a + b + c
         11
         12
        13 def say_hello(name, *, formal = True):
        14     return f"Pleased to meet you, {name}" if formal else f"Hi, {name}!"
        15
        16
        17 exec_add = execute(add)
        18 exec_greet = execute(say_hello)
```

```
In [320]: 1 exec_add(1, 2, 3)
```

Out[320]: 6

```
In [321]: 1 exec_greet("Michael", formal = False)
```

Out[321]: 'Hi, Michael!'

```
In [322]: 1 exec_greet("Michael", formal = True)
```

Out[322]: 'Pleased to meet you, Michael'

```
In [323]: 1 print((0.1).as_integer_ratio())
```

(3602879701896397, 36028797018963968)

```
In [324]: 1 print((0.125).as_integer_ratio())
```

(1, 8)

```
In [325]: 1 print((0.2).as_integer_ratio())
          2 print(f"{0.2:.40f}")
```

```
(3602879701896397, 18014398509481984)
0.2000000000000000111022302462515654042363
```

```
In [326]: 1 print(math.pi.as_integer_ratio())
          2 n, d = math.pi.as_integer_ratio()
          3 print(n / d)
```

```
(884279719003555, 281474976710656)
3.141592653589793
```

```
In [327]: 1 try:
          2     a1 = a1 + 1
          3     print(a1)
          4 except NameError as ex:
          5     print(type(ex).      , ": ", *ex.args, ".", sep = "")
```

```
NameError: name 'a1' is not defined.
```

```
In [328]: 1 def getDiagonalMatrix(val, rows, columns):
          2     diagonalMatrix = [[val if ce == r else 0 for ce in range(4)] for r in
          3     range(5)]
          4     return diagonalMatrix
```

```
In [329]: 1 def getDiagonalMatrix(rows, columns, *, val):
          2     diagonalMatrix = [[val if cl == row else 0 for cl in range(columns)]
          3     for row in range(rows)]
          4     return diagonalMatrix
```

```
In [330]: 1 def getFactorial(n):
          2     factorial = 1
          3     for i in range(1, n + 1):
          4         factorial *= i
          5     return factorial
```

```
In [331]: 1 result = getFactorial(5)
          2 print(result)
```

```
120
```

```
In [332]: 1 result = getDiagonalMatrix(3, 4, val = 5)
          2 pprint.pprint(result, width = 14)
```

```
[[5, 0, 0, 0],
 [0, 5, 0, 0],
 [0, 0, 5, 0]]
```

```
In [333]: 1 print(f"{timeit.timeit('getFactorial(2)', globals = globals()):.4g}")
```

```
0.1758
```

In [334]:

```
1 n = 0.000000000000123456789
2 n = 4.482882e-05
3 exp = 0
4 while n < 1:
5     exp -= 3
6     n *= 1000
7 prefixes = "", "m", "\N{GREEK SMALL LETTER MU}", "n", "p", "f", "a"
8 print(f"{n}{prefixes[abs(exp)//3]}")
```

44.82882μ

In [335]:

```
1 n = 1000000000
2 exp = 0
3 while n >= 1000:
4     exp += 3
5     n /= 1000
6 prefixes = "", "k", "M", "G", "T", "P", "E"
7 print(f"{n}{prefixes[exp//3]}")
```

1.0G

In [7]:

```
def toEnggNotation(n, scientificNotation = True, nSignificantDigits = 4):
1     exp = 0
2     oldN = n
3     if n > 1:
4         while n >= 1000:
5             exp += 3
6             n /= 1000
7             prefixes = "", "k", "M", "G", "T", "P", "E"
8             return f"{n:.3f}{prefixes[exp//3]}\t{oldN}" if
scientificNotation else f"{n}{prefixes[exp//3]}"
9         elif n < 1:
10            while n < 1:
11                exp -= 3
12                n *= 1000
13                prefixes = "", "m", "\N{GREEK SMALL LETTER MU}", "n", "p", "f",
14                "a"
15            return f"{n:.{nSignificantDigits}f}
16            {prefixes[abs(exp)//3]}\t{oldN}" if scientificNotation else f"{n}
17            {prefixes[abs(exp)//3]}"
18        else:
19            return 1
```

```
1 def myTimer(f, *pos, **keyword):
2     start = time.perf_counter()
3     print(f(*pos, **keyword))
4     end = time.perf_counter()
5     return end - start
6
7
```

```
8 print(toEnggNotation(myTimer(getFactorial, 1000)))
9 print(toEnggNotation(myTimer(getDiagonalMatrix, 3, 5, val = 7)))
```

[illegible]

```

1 def encloseMyTimer(f):
2
3     def myTimer(*pos, **keyword):
4         start = time.perf_counter()
5         result = f(*pos, **keyword)
6         end = time.perf_counter()
7         print(toEnggNotation(end - start, True, 4))
8         return result
9
10    return myTimer
11
12
13 timed = encloseMyTimer(getDiagonalMatrix)
14 print(timed(2, 4, val = -5))
15 timed = encloseMyTimer(getFactorial)
16 print(timed(11))
17 print(encloseMyTimer(getDiagonalMatrix)(2, 4, val = -5))
18 print(encloseMyTimer(getFactorial)(11))

```

```
5.2100µ 5.209999983435409e-06
[[-5, 0, 0, 0], [0, -5, 0, 0]]
4.1080µ 4.108000041469495e-06
39916800
3.6170µ 3.6170000043966866e-06
[[-5, 0, 0, 0], [0, -5, 0, 0]]
2.4240µ 2.4240000016106933e-06
39916800
```

```
In [341]: 1 # print(eval("2 * 3"))
          2 # print(eval("x = 2 and print(x)"))
```

```
In [342]: 1 # eval("x = 2")
          2 # print(x)
```

```
In [343]: 1 x = 2
          2 eval("x ** 2 - 1")
```

```
Out[343]: 3
```

```
In [344]: 1 def f(expr, x):
          2     print(eval(expr))
          3
          4
          5 x = 2
          6 f("x ** 2 - 1", x)
```

```
3
```

```
In [345]: 1 def f(x):
          2     return True
          3
          4
          5 try:
          6     print(list(map(f(False), range(0, 5))))
          7 except TypeError as ex:
          8     print(type(ex).      + ":", ex.args[0])
```

```
TypeError: 'bool' object is not callable
```

```
In [346]: 1 def getExtremum(expr, interval, granularity, minimax):
          2
          3     def f(expr):
          4
          5         def g(x):
          6             return eval(expr)
          7
          8         return g
          9
          10     evalExpr = f(expr)
          11     print("Range:", list(range(interval[0], interval[1] + granularity,
          12 granularity)))
          13     if minimax == "minimum":
```

```

7         return min(map(evalExpr, range(interval[0], interval[1] +
granularity, granularity)))
8     elif minimax == "maximum":
9         return max(map(evalExpr, range(interval[0], interval[1] +
granularity, granularity)))
10    else:
11        return "Invalid minimax"
12
13    expr = "x ** 2 - 1"
14    print(getExtremum(expr, (-10, 10), 2, "minimum"))
15    print(getExtremum(expr, (-10, 10), 2, "maximum"))
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

```

Range: [-10, -8, -6, -4, -2, 0, 2, 4, 6, 8, 10]
-1
Range: [-10, -8, -6, -4, -2, 0, 2, 4, 6, 8, 10]
99

```

```

In [347]: 1 expr = "x ** 2 - 1"
          2 f = lambda x: eval(expr)
          3 print(f(3))

```

8

```

In [348]: 1 def getExtremum(expr, interval, resolution, minimax):
          2     def f(expr):
          3         def g(x):
          4             return eval(expr)
          5         return g
          6     evalExpr = f(expr)

```

```

    l = [interval[0] + (i * (interval[1] - interval[0]) / (resolution -
6    1)) for i in range(resolution)] # Cheated with the calc.
        return min(map(evalExpr, l)) if minimax == "minimum" else
7    max(map(evalExpr, l))

8
    expr = "x ** 2 - 1"
9    print(getExtremum(expr, (-10, 10), 5, "minimum"))
1    print(getExtremum(expr, (-10, 10), 5, "maximum"))
0    print(getExtremum(expr, (-10, 10), 6, "minimum"))
1    print(getExtremum(expr, (-10, 10), 6, "maximum"))
1    expr = "math.log10(x)"
1    print(getExtremum(expr, (1, 100), 20, "minimum"))
2    print(getExtremum(expr, (1, 100), 20, "maximum"))
1    expr = "math.sin(x)"
3    print(getExtremum(expr, (-10, 10), 20, "minimum"))
1    print(getExtremum(expr, (-10, 10), 20000, "maximum"))
4    expr = "x"
1    print(getExtremum(expr, (1, 6), 8, "minimum"))
5    print(getExtremum(expr, (1, 6), 8, "maximum"))

1
6
1
7
1
8
1
9
2
0
2
1
2
2
2
3
2
4
2
5
2
6
2
7
2
8

```

```
-1.0
99.0
3.0
99.0
0.0
2.0
-0.9999667804441291
0.999999996044897
1.0
6.0
```

In [349]:

```
def f(t):
    return t[0] ** t[1]

l = [(1, 1), (1, 2), (2, 1), (2, 2), (2, 3), (3, 2), (3, 3), (3, 4), (4, 3),
      (4, 4)]

start = time.perf_counter()
for _ in range(10_000):
    result = []
    for t in l:
        result.append(f(t))
end = time.perf_counter()
forLoopTime = end - start
print(result)

start = time.perf_counter()
for _ in range(10_000):
    result = [f(t) for t in l]
end = time.perf_counter()
listCompTime = end - start
print(result)

start = time.perf_counter()
for _ in range(10_000):
    result = list(map(f, l))
end = time.perf_counter()
mapTime = end - start
print(result)

print(toEnggNotation(forLoopTime), toEnggNotation(listCompTime),
      toEnggNotation(mapTime))
```

2
2
2
3
2
4
2
5
2
6
2
7
2
8
2
9
3
0

```
[1, 1, 2, 4, 8, 9, 27, 81, 64, 256]
[1, 1, 2, 4, 8, 9, 27, 81, 64, 256]
[1, 1, 2, 4, 8, 9, 27, 81, 64, 256]
15.1299m      0.0151298600000001827  10.1019m   0.010101901999973961  10.2137m   0.010213654999972732
```

In [350]:

```
def power(x, y):
    return x ** y

def dist(pt1, pt2):
    return math.sqrt(sum(coord1 - coord2 for coord1, coord2 in zip(pt1,
pt2)))

def partial(f, operand2):

    def calcPartialFunction(operand1):
        return f(operand1, operand2)

    return calcPartialFunction

squares = partial(power, 2)
print(squares(3))
print(squares(4))
dist_from_origin = partial(dist, (0, 0))
print(dist_from_origin((1, 1)))
gcd_60 = partial(math.gcd, 60)
print(gcd_60(13))
log_2 = partial(math.log, 2)
print(log_2(8))
log_10 = partial(math.log, 10)
print(log_10(100))
```

```
5 log_16 = partial(math.log, 16)
6 print(log_16(65536))
```

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
```

```
9
16
1.4142135623730951
1
3.0
2.0
4.0
```

In [351]:

```
1 def norm(x, y):
2     return math.sqrt(x ** 2 + y ** 2)
3
4
5 def find_index_min(seq):
6     minima = min(seq)
7     return seq.index(minima)
8
9
10 def logged(f):
11
12     def g(*args):
13         print("Logging...")
14         start = time.perf_counter()
15         print(f(*args))
```

```

16         end = time.perf_counter()
17         print(toEnggNotation(end - start))
18         print("... Finished")
19
20     return g
21
22
23 loggedNorm = logged(norm)
24 loggedNorm(3, 4)
25 loggedFindIndexMin = logged(find_index_min)
26 loggedFindIndexMin([8, 7, 2, 9, 4])

```

```

Logging...
5.0
16.9020µ      1.6901999970286852e-05
... Finished
Logging...
2
12.1130µ      1.2113000025237852e-05
... Finished

```

```

In [352]: 1 def encloseGetPower(f):
2
3           def getPower(*args):
4               return f(*args)
5
6           print(hex(id(getPower)))
7           return getPower
8
9
10 getPower = encloseGetPower(pow)
11 print(getPower(2, 5))
12 print(hex(id(getPower)))
13 print(hex(id(encloseGetPower)))
14 print(getPower.          )
15 print(encloseGetPower.   )

```

```

0x74efc24f9430
32
0x74efc24f9430
0x74efc24f94e0
(<cell at 0x74efc5818ac0: builtin_function_or_method object at 0x74f01fbb0630>,)
None

```

```

In [353]: 1 l = [(1, 2), (-4, -5.5), (10, -10), (-2, 2)]
2          print(sorted(l, key = lambda t: abs(t[0] - t[1]), reverse = True))

```

```

[(10, -10), (-2, 2), (-4, -5.5), (1, 2)]

```

```

In [354]: 1 l = ["s", "0", "1"]
2          card = "".join(reversed(l))
3          print(card)
4          card = ""
5          for k in reversed(l):

```

```
6     card += k
7     print(card)
```

```
10s
10s
```

In [355]:

```
1
2
3
4
5 suits = 'shdc' # Spades, Hearts, Diamonds, Clubs
6 ranks = list('23456789') + ['10', 'J', 'Q', 'K', 'A']
7 cards = [[rank + suit for rank in ranks] for suit in suits]
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
```

```
[['As', 'Ks', 'Qs', 'Js', '10s', '9s', '8s', '7s', '6s', '5s', '4s', '3s', '2s'], ['Ah', 'Kh', 'Qh', 'Jh', '10h', '9h', '8h', '7h', '6h', '5h', '4h', '3h', '2h'], ['Ad', 'Kd', 'Qd', 'Jd', '10d', '9d', '8d', '7d', '6d', '5d', '4d', '3d', '2d'], ['Ac', 'Kc', 'Qc', 'Jc', '10c', '9c', '8c', '7c', '6c', '5c', '4c', '3c', '2c']]
[['2s', '3s', '4s', '5s', '6s', '7s', '8s', '9s', '10s', 'Js', 'Qs', 'Ks', 'As'], ['2h', '3h', '4h', '5h', '6h', '7h', '8h', '9h', '10h', 'Jh', 'Qh', 'Kh', 'Ah'], ['2d', '3d', '4d', '5d', '6d', '7d', '8d', '9d', '10d', 'Jd', 'Qd', 'Kd', 'Ad'], ['2c', '3c', '4c', '5c', '6c', '7c', '8c', '9c', '10c', 'Jc', 'Qc', 'Kc', 'Ac']]
```

In [356]:

```
1 data = {'S1': (100, 200, 80, 180), 'S2': (10, 20, 8, 18), 'S3': (50, 150, 50, 150)}
```

```

2
3 def findMinima(data, reverse = False):
4     minimax = min if reverse else max
5     return minimax(data, key = lambda d: data[d][1] - data[d][2])
6
7
8
9
10 print(findMinima(data, reverse = True))
11 print(findMinima(data))
12

```

S2
S1

In [357]:

```

1 quotes = [('AACC', 6.05, 6.07, 6.03, 6.05, 65800), ('AAME', 1.7, 1.82, 1.7,
2 1.82, 4300), ('AAON', 24.98, 25.07, 24.9, 24.94, 28200), ('AAPL', 317.99,
3 319.57, 316.75, 317.13, 12901800), ('AATI', 3.82, 3.82, 3.74, 3.79, 194600),
4 ('AAWW', 60.89, 61.44, 60.5, 61.19, 272800), ('AAXJ', 65.4, 65.71, 65.28,
5 65.56, 390300), ('ABAT', 4.01, 4.01, 3.95, 3.99, 656300), ('ABAX', 25.26,
6 25.49, 25.04, 25.42, 73700), ('ABBC', 11.75, 11.88, 11.48, 11.53, 29700),
7 ('ABCB', 9.3, 9.3, 9.06, 9.14, 42600), ('ABCD', 3.25, 3.25, 3.11, 3.22,
8 122800), ('ABCO', 48.75, 50.41, 46.9, 50.37, 66300), ('ABCW', 0.52, 0.61, 0.52,
9 0.53, 83000), ('ABFS', 25.98, 26.27, 25.41, 25.5, 384900), ('ABIO', 3.96, 4,
10 3.88, 4, 38500), ('ABMD', 11.94, 12, 11.69, 11.87, 122600), ('ABTL', 0.82,
11 0.84, 0.82, 0.83, 28700), ('ABVA', 3.09, 3.25, 3.09, 3.25, 6200), ('ACAD',
12 0.76, 0.76, 0.7, 0.74, 341500), ('ACAS', 7.52, 7.72, 7.52, 7.66, 5199800),
13 ('ACAT', 14.44, 14.44, 14.04, 14.2, 51700), ('ACCL', 8.11, 8.21, 7.94, 8.1,
14 456100), ('ACET', 8.01, 8.04, 7.13, 7.73, 575600), ('ACFC', 1.69, 1.7, 1.5,
15 1.6, 12300), ('ACFN', 3.82, 4, 3.82, 3.98, 53700), ('ACGL', 89.76, 90.14,
16 89.39, 89.92, 240900), ('ACGY', 22.41, 22.56, 22.25, 22.46, 86800), ('ACHN',
17 3.12, 3.2, 3.07, 3.16, 113700), ('ACIW', 26.96, 27.03, 26.63, 26.8, 157000),
18 ('ACLI', 33.65, 33.77, 33.45, 33.63, 28700), ('ACLS', 2.47, 2.63, 2.46, 2.53,
19 1818800), ('ACMR', 2.69, 2.84, 2.37, 2.71, 158600), ('ACOM', 25.2, 26.6, 24.9,
20 26.56, 265300), ('ACOR', 26.67, 27.07, 26.38, 27.04, 1415000), ('ACPW', 1.84,
21 1.89, 1.77, 1.85, 565500), ('ACTG', 27.2, 27.43, 26.86, 27.18, 228800),
22 ('ACTI', 3.25, 3.26, 3.25, 3.26, 148500), ('ACTS', 2.08, 2.09, 2.07, 2.07,
23 130500), ('ACUR', 2.6, 2.64, 2.51, 2.6, 16000), ('ACWI', 46.53, 46.7, 46.32,
24 46.51, 286200), ('ACWX', 44.49, 44.66, 44.36, 44.6, 55500), ('ACXM', 18, 18.07,
25 17.81, 18.01, 289800), ('ADAM', 7.34, 7.49, 7.33, 7.44, 81700), ('ADAT', 0.6,
26 0.68, 0.59, 0.66, 86400), ('ADBE', 29.43, 29.71, 29.07, 29.14, 7585300),
27 ('ADCT', 12.68, 12.69, 12.66, 12.68, 1660500), ('ADEP', 6.14, 6.14, 4.95, 5.61,
28 71000), ('ADES', 6.2, 6.22, 6, 6.19, 4800), ('ADGF', 4.31, 4.55, 4.31, 4.54,
29 10200)]

```

```
def myFilter(percentage = 10):
    return list(filter(lambda t: abs((t[4] - t[2]) / t[2]) > (percentage / 100), quotes))

print("10:", myFilter(10))
print("5:", myFilter(5))
print("1:", myFilter(1))
```

```
10: [('ABCW', 0.52, 0.61, 0.52, 0.53, 83000)]
5: [('ABCW', 0.52, 0.61, 0.52, 0.53, 83000), ('ACFC', 1.69, 1.7, 1.5, 1.6, 12300), ('ADEP', 6.14, 6.14, 4.95, 5.61, 71000)]
1: [('ABBC', 11.75, 11.88, 11.48, 11.53, 29700), ('ABCB', 9.3, 9.3, 9.06, 9.14, 42600), ('ABCW', 0.52, 0.61, 0.52, 0.53, 83000), ('ABFS', 25.98, 26.27, 25.41, 25.5, 384900), ('ABMD', 11.94, 12, 11.69, 11.87, 122600), ('ABTL', 0.82, 0.84, 0.82, 0.83, 28700), ('ACAD', 0.76, 0.76, 0.7, 0.74, 341500), ('ACAT', 14.44, 14.44, 14.04, 14.2, 51700), ('ACCL', 8.11, 8.21, 7.94, 8.1, 456100), ('ACET', 8.01, 8.04, 7.13, 7.73, 575600), ('ACFC', 1.69, 1.7, 1.5, 1.6, 12300), ('ACHN', 3.12, 3.2, 3.07, 3.16, 113700), ('ACLS', 2.47, 2.63, 2.46, 2.53, 1818800), ('ACMR', 2.69, 2.84, 2.37, 2.71, 158600), ('ACPW', 1.84, 1.89, 1.77, 1.85, 565500), ('ACUR', 2.6, 2.64, 2.51, 2.6, 16000), ('ADAT', 0.6, 0.68, 0.59, 0.66, 86400), ('ADBE', 29.43, 29.71, 29.07, 29.14, 7585300), ('ADEP', 6.14, 6.14, 4.95, 5.61, 71000)]
```

```
In [358]: 1 l = [5, 6, -4, 8]
          2 print(min(l, key = abs))
```

```
-4
```

```
In [462]: 1 importlib.reload(logging)
          2 logging.basicConfig(format = "%(asctime)s %(levelname)s %(message)s", datefmt = "%FT%T")
          3 logging.error("This is an error")
```

```
2026-09-18T20:52:36 ERROR This is an error
```

```
In [360]: 1 logger = logging.getLogger(
          2 logger.setLevel(logging.DEBUG)
          3 logger.error("This is an error message")
          4 logger.debug("This is a debug message")
```

```
2026-09-18T17:28:00b ERROR This is an error message
2026-09-18T17:28:00b DEBUG This is a debug message
```

```
In [361]: 1 def f():
          2     pass
          3
          4
          5 print(f.
          6 print(type(f.))
```

```
f
<class 'str'>
```

```
In [362]: 1 def partial(f):
          2
          3     def callF(*pargs, **kwargs):
          4         logger.debug("Calling your function... " + f. )
```

```

5         start = time.perf_counter()
6         result = f(*pargs, **kwargs)
7         end = time.perf_counter()
8         logger.debug("... Function returned.")
9         logger.info(f"Elapsed Time: {toEnggNotation(end - start)}")
10        # print(f"{toEnggNotation(end - start)}")
11        return result
12
13    return callF
14
15
16    def add(a, b):
17        return a + b
18
19
20    def log3(a):
21        return math.log(a, 3)
22
23
24    @partial
25    def log4(a):
26        return math.log(a, 4)
27
28
29    myAdder = partial(add)
30    logger.info("myAdder: " + str(myAdder(2, 3)))
31    log3 = partial(log3)
32    logger.info("log3: " + str(log3(9)))
33    logger.info("log4: " + str(log4(16)))

```

```

2026-09-18T17:28:00b DEBUG Calling your function... add
2026-09-18T17:28:00b DEBUG ... Function returned.
2026-09-18T17:28:00b INFO Elapsed Time: 1.5930µ 1.5930000358821417e-06
2026-09-18T17:28:00b INFO myAdder: 5
2026-09-18T17:28:00b DEBUG Calling your function... log3
2026-09-18T17:28:00b DEBUG ... Function returned.
2026-09-18T17:28:00b INFO Elapsed Time: 20.6290µ 2.0628999948257842e-05
2026-09-18T17:28:00b INFO log3: 2.0
2026-09-18T17:28:00b DEBUG Calling your function... log4
2026-09-18T17:28:00b DEBUG ... Function returned.
2026-09-18T17:28:00b INFO Elapsed Time: 4.6590µ 4.659000012452452e-06
2026-09-18T17:28:00b INFO log4: 2.0

```

In [363]:

```

1    def cache(f):
2        print("Initializing cache...")
3        cacheDict = {}
4
5        def callF(*pargs):
6            if pargs in cacheDict:
7                print("Cache Hit")
8                return cacheDict[pargs]
9            else:
10               print("Cache Miss")
11               result = f(*pargs)

```

```

12         cacheDict[pargs] = result
13         return result
14
15     return callF
16
17
18 @cache
19 def add(a, b):
20     return a + b
21
22
23 @cache
24 def mul(a, b):
25     return a * b

```

Initializing cache...

Initializing cache...

```

In [364]: 1 print(add.      )
          2 print(mul.    )

```

(<cell at 0x74efc252cdf0: dict object at 0x74efc253fb00>, <cell at 0x74f01f194070: function object at 0x74efc2537b60>)

(<cell at 0x74efc252e310: dict object at 0x74efc253fb40>, <cell at 0x74efc252f220: function object at 0x74efc2537d70>)

```

In [365]: 1 print(add(2, 3))
          2 print(mul(2, 3))
          3 print(add(2, 3))
          4 print(mul(2, 3))

```

Cache Miss

5

Cache Miss

6

Cache Hit

5

Cache Hit

6

```

In [366]: 1 print(type(func tools.lru_cache))

```

<class 'function'>

```

In [367]: 1 @func tools.lru_cache(maxsize = 2)
          2 def graham(a, b, c):
          3     print("Cache Miss")
          4     return a ** b ** c
          5
          6
          7 print(graham(2, 2, 4))

```

Cache Miss

65536

```
In [368]: 1 print(gramham(2, 2, 4))
          2 print(gramham(2, 2, 4))
          3 print(gramham(2, 2, 5))
          4 print(gramham(2, 2, 6))
          5 print(gramham(2, 2, 4))
```

```
65536
65536
Cache Miss
4294967296
Cache Miss
18446744073709551616
Cache Miss
65536
```

```
In [369]: 1 @functools.lru_cache(maxsize = 100)
          2 def fib(n):
          3     print(f"Calling fib{n}...")
          4     if n <= 1:
          5         return n
          6     return fib(n - 1) + fib(n - 2)
          7
          8
          9 print(fib(2000))
```

Calling fib2000...
Calling fib1999...
Calling fib1998...
Calling fib1997...
Calling fib1996...
Calling fib1995...
Calling fib1994...
Calling fib1993...
Calling fib1992...
Calling fib1991...
Calling fib1990...
Calling fib1989...
Calling fib1988...
Calling fib1987...
Calling fib1986...
Calling fib1985...
Calling fib1984...
Calling fib1983...
Calling fib1982...
Calling fib1981...
Calling fib1980...
Calling fib1979...
Calling fib1978...
Calling fib1977...
Calling fib1976...
Calling fib1975...
Calling fib1974...
Calling fib1973...
Calling fib1972...
Calling fib1971...
Calling fib1970...
Calling fib1969...
Calling fib1968...
Calling fib1967...
Calling fib1966...
Calling fib1965...
Calling fib1964...
Calling fib1963...
Calling fib1962...
Calling fib1961...
Calling fib1960...
Calling fib1959...
Calling fib1958...
Calling fib1957...
Calling fib1956...
Calling fib1955...
Calling fib1954...
Calling fib1953...
Calling fib1952...
Calling fib1951...
Calling fib1950...
Calling fib1949...
Calling fib1948...
Calling fib1947...
Calling fib1946...
Calling fib1945...
Calling fib1944...
Calling fib1943...
Calling fib1942...
Calling fib1941...
Calling fib1940...

Calling fib1939...
Calling fib1938...
Calling fib1937...
Calling fib1936...
Calling fib1935...
Calling fib1934...
Calling fib1933...
Calling fib1932...
Calling fib1931...
Calling fib1930...
Calling fib1929...
Calling fib1928...
Calling fib1927...
Calling fib1926...
Calling fib1925...
Calling fib1924...
Calling fib1923...
Calling fib1922...
Calling fib1921...
Calling fib1920...
Calling fib1919...
Calling fib1918...
Calling fib1917...
Calling fib1916...
Calling fib1915...
Calling fib1914...
Calling fib1913...
Calling fib1912...
Calling fib1911...
Calling fib1910...
Calling fib1909...
Calling fib1908...
Calling fib1907...
Calling fib1906...
Calling fib1905...
Calling fib1904...
Calling fib1903...
Calling fib1902...
Calling fib1901...
Calling fib1900...
Calling fib1899...
Calling fib1898...
Calling fib1897...
Calling fib1896...
Calling fib1895...
Calling fib1894...
Calling fib1893...
Calling fib1892...
Calling fib1891...
Calling fib1890...
Calling fib1889...
Calling fib1888...
Calling fib1887...
Calling fib1886...
Calling fib1885...
Calling fib1884...
Calling fib1883...
Calling fib1882...
Calling fib1881...
Calling fib1880...
Calling fib1879...

Calling fib1878...
Calling fib1877...
Calling fib1876...
Calling fib1875...
Calling fib1874...
Calling fib1873...
Calling fib1872...
Calling fib1871...
Calling fib1870...
Calling fib1869...
Calling fib1868...
Calling fib1867...
Calling fib1866...
Calling fib1865...
Calling fib1864...
Calling fib1863...
Calling fib1862...
Calling fib1861...
Calling fib1860...
Calling fib1859...
Calling fib1858...
Calling fib1857...
Calling fib1856...
Calling fib1855...
Calling fib1854...
Calling fib1853...
Calling fib1852...
Calling fib1851...
Calling fib1850...
Calling fib1849...
Calling fib1848...
Calling fib1847...
Calling fib1846...
Calling fib1845...
Calling fib1844...
Calling fib1843...
Calling fib1842...
Calling fib1841...
Calling fib1840...
Calling fib1839...
Calling fib1838...
Calling fib1837...
Calling fib1836...
Calling fib1835...
Calling fib1834...
Calling fib1833...
Calling fib1832...
Calling fib1831...
Calling fib1830...
Calling fib1829...
Calling fib1828...
Calling fib1827...
Calling fib1826...
Calling fib1825...
Calling fib1824...
Calling fib1823...
Calling fib1822...
Calling fib1821...
Calling fib1820...
Calling fib1819...
Calling fib1818...

Calling fib1817...
Calling fib1816...
Calling fib1815...
Calling fib1814...
Calling fib1813...
Calling fib1812...
Calling fib1811...
Calling fib1810...
Calling fib1809...
Calling fib1808...
Calling fib1807...
Calling fib1806...
Calling fib1805...
Calling fib1804...
Calling fib1803...
Calling fib1802...
Calling fib1801...
Calling fib1800...
Calling fib1799...
Calling fib1798...
Calling fib1797...
Calling fib1796...
Calling fib1795...
Calling fib1794...
Calling fib1793...
Calling fib1792...
Calling fib1791...
Calling fib1790...
Calling fib1789...
Calling fib1788...
Calling fib1787...
Calling fib1786...
Calling fib1785...
Calling fib1784...
Calling fib1783...
Calling fib1782...
Calling fib1781...
Calling fib1780...
Calling fib1779...
Calling fib1778...
Calling fib1777...
Calling fib1776...
Calling fib1775...
Calling fib1774...
Calling fib1773...
Calling fib1772...
Calling fib1771...
Calling fib1770...
Calling fib1769...
Calling fib1768...
Calling fib1767...
Calling fib1766...
Calling fib1765...
Calling fib1764...
Calling fib1763...
Calling fib1762...
Calling fib1761...
Calling fib1760...
Calling fib1759...
Calling fib1758...
Calling fib1757...

Calling fib1756...
Calling fib1755...
Calling fib1754...
Calling fib1753...
Calling fib1752...
Calling fib1751...
Calling fib1750...
Calling fib1749...
Calling fib1748...
Calling fib1747...
Calling fib1746...
Calling fib1745...
Calling fib1744...
Calling fib1743...
Calling fib1742...
Calling fib1741...
Calling fib1740...
Calling fib1739...
Calling fib1738...
Calling fib1737...
Calling fib1736...
Calling fib1735...
Calling fib1734...
Calling fib1733...
Calling fib1732...
Calling fib1731...
Calling fib1730...
Calling fib1729...
Calling fib1728...
Calling fib1727...
Calling fib1726...
Calling fib1725...
Calling fib1724...
Calling fib1723...
Calling fib1722...
Calling fib1721...
Calling fib1720...
Calling fib1719...
Calling fib1718...
Calling fib1717...
Calling fib1716...
Calling fib1715...
Calling fib1714...
Calling fib1713...
Calling fib1712...
Calling fib1711...
Calling fib1710...
Calling fib1709...
Calling fib1708...
Calling fib1707...
Calling fib1706...
Calling fib1705...
Calling fib1704...
Calling fib1703...
Calling fib1702...
Calling fib1701...
Calling fib1700...
Calling fib1699...
Calling fib1698...
Calling fib1697...
Calling fib1696...

Calling fib1695...
Calling fib1694...
Calling fib1693...
Calling fib1692...
Calling fib1691...
Calling fib1690...
Calling fib1689...
Calling fib1688...
Calling fib1687...
Calling fib1686...
Calling fib1685...
Calling fib1684...
Calling fib1683...
Calling fib1682...
Calling fib1681...
Calling fib1680...
Calling fib1679...
Calling fib1678...
Calling fib1677...
Calling fib1676...
Calling fib1675...
Calling fib1674...
Calling fib1673...
Calling fib1672...
Calling fib1671...
Calling fib1670...
Calling fib1669...
Calling fib1668...
Calling fib1667...
Calling fib1666...
Calling fib1665...
Calling fib1664...
Calling fib1663...
Calling fib1662...
Calling fib1661...
Calling fib1660...
Calling fib1659...
Calling fib1658...
Calling fib1657...
Calling fib1656...
Calling fib1655...
Calling fib1654...
Calling fib1653...
Calling fib1652...
Calling fib1651...
Calling fib1650...
Calling fib1649...
Calling fib1648...
Calling fib1647...
Calling fib1646...
Calling fib1645...
Calling fib1644...
Calling fib1643...
Calling fib1642...
Calling fib1641...
Calling fib1640...
Calling fib1639...
Calling fib1638...
Calling fib1637...
Calling fib1636...
Calling fib1635...

Calling fib1634...
Calling fib1633...
Calling fib1632...
Calling fib1631...
Calling fib1630...
Calling fib1629...
Calling fib1628...
Calling fib1627...
Calling fib1626...
Calling fib1625...
Calling fib1624...
Calling fib1623...
Calling fib1622...
Calling fib1621...
Calling fib1620...
Calling fib1619...
Calling fib1618...
Calling fib1617...
Calling fib1616...
Calling fib1615...
Calling fib1614...
Calling fib1613...
Calling fib1612...
Calling fib1611...
Calling fib1610...
Calling fib1609...
Calling fib1608...
Calling fib1607...
Calling fib1606...
Calling fib1605...
Calling fib1604...
Calling fib1603...
Calling fib1602...
Calling fib1601...
Calling fib1600...
Calling fib1599...
Calling fib1598...
Calling fib1597...
Calling fib1596...
Calling fib1595...
Calling fib1594...
Calling fib1593...
Calling fib1592...
Calling fib1591...
Calling fib1590...
Calling fib1589...
Calling fib1588...
Calling fib1587...
Calling fib1586...
Calling fib1585...
Calling fib1584...
Calling fib1583...
Calling fib1582...
Calling fib1581...
Calling fib1580...
Calling fib1579...
Calling fib1578...
Calling fib1577...
Calling fib1576...
Calling fib1575...
Calling fib1574...

Calling fib1573...
Calling fib1572...
Calling fib1571...
Calling fib1570...
Calling fib1569...
Calling fib1568...
Calling fib1567...
Calling fib1566...
Calling fib1565...
Calling fib1564...
Calling fib1563...
Calling fib1562...
Calling fib1561...
Calling fib1560...
Calling fib1559...
Calling fib1558...
Calling fib1557...
Calling fib1556...
Calling fib1555...
Calling fib1554...
Calling fib1553...
Calling fib1552...
Calling fib1551...
Calling fib1550...
Calling fib1549...
Calling fib1548...
Calling fib1547...
Calling fib1546...
Calling fib1545...
Calling fib1544...
Calling fib1543...
Calling fib1542...
Calling fib1541...
Calling fib1540...
Calling fib1539...
Calling fib1538...
Calling fib1537...
Calling fib1536...
Calling fib1535...
Calling fib1534...
Calling fib1533...
Calling fib1532...
Calling fib1531...
Calling fib1530...
Calling fib1529...
Calling fib1528...
Calling fib1527...
Calling fib1526...
Calling fib1525...
Calling fib1524...
Calling fib1523...
Calling fib1522...
Calling fib1521...
Calling fib1520...
Calling fib1519...
Calling fib1518...
Calling fib1517...
Calling fib1516...
Calling fib1515...
Calling fib1514...
Calling fib1513...

Calling fib1512...
Calling fib1511...
Calling fib1510...
Calling fib1509...
Calling fib1508...
Calling fib1507...
Calling fib1506...
Calling fib1505...
Calling fib1504...
Calling fib1503...
Calling fib1502...
Calling fib1501...
Calling fib1500...
Calling fib1499...
Calling fib1498...
Calling fib1497...
Calling fib1496...
Calling fib1495...
Calling fib1494...
Calling fib1493...
Calling fib1492...
Calling fib1491...
Calling fib1490...
Calling fib1489...
Calling fib1488...
Calling fib1487...
Calling fib1486...
Calling fib1485...
Calling fib1484...
Calling fib1483...
Calling fib1482...
Calling fib1481...
Calling fib1480...
Calling fib1479...
Calling fib1478...
Calling fib1477...
Calling fib1476...
Calling fib1475...
Calling fib1474...
Calling fib1473...
Calling fib1472...
Calling fib1471...
Calling fib1470...
Calling fib1469...
Calling fib1468...
Calling fib1467...
Calling fib1466...
Calling fib1465...
Calling fib1464...
Calling fib1463...
Calling fib1462...
Calling fib1461...
Calling fib1460...
Calling fib1459...
Calling fib1458...
Calling fib1457...
Calling fib1456...
Calling fib1455...
Calling fib1454...
Calling fib1453...
Calling fib1452...

Calling fib1451...
Calling fib1450...
Calling fib1449...
Calling fib1448...
Calling fib1447...
Calling fib1446...
Calling fib1445...
Calling fib1444...
Calling fib1443...
Calling fib1442...
Calling fib1441...
Calling fib1440...
Calling fib1439...
Calling fib1438...
Calling fib1437...
Calling fib1436...
Calling fib1435...
Calling fib1434...
Calling fib1433...
Calling fib1432...
Calling fib1431...
Calling fib1430...
Calling fib1429...
Calling fib1428...
Calling fib1427...
Calling fib1426...
Calling fib1425...
Calling fib1424...
Calling fib1423...
Calling fib1422...
Calling fib1421...
Calling fib1420...
Calling fib1419...
Calling fib1418...
Calling fib1417...
Calling fib1416...
Calling fib1415...
Calling fib1414...
Calling fib1413...
Calling fib1412...
Calling fib1411...
Calling fib1410...
Calling fib1409...
Calling fib1408...
Calling fib1407...
Calling fib1406...
Calling fib1405...
Calling fib1404...
Calling fib1403...
Calling fib1402...
Calling fib1401...
Calling fib1400...
Calling fib1399...
Calling fib1398...
Calling fib1397...
Calling fib1396...
Calling fib1395...
Calling fib1394...
Calling fib1393...
Calling fib1392...
Calling fib1391...

Calling fib1390...
Calling fib1389...
Calling fib1388...
Calling fib1387...
Calling fib1386...
Calling fib1385...
Calling fib1384...
Calling fib1383...
Calling fib1382...
Calling fib1381...
Calling fib1380...
Calling fib1379...
Calling fib1378...
Calling fib1377...
Calling fib1376...
Calling fib1375...
Calling fib1374...
Calling fib1373...
Calling fib1372...
Calling fib1371...
Calling fib1370...
Calling fib1369...
Calling fib1368...
Calling fib1367...
Calling fib1366...
Calling fib1365...
Calling fib1364...
Calling fib1363...
Calling fib1362...
Calling fib1361...
Calling fib1360...
Calling fib1359...
Calling fib1358...
Calling fib1357...
Calling fib1356...
Calling fib1355...
Calling fib1354...
Calling fib1353...
Calling fib1352...
Calling fib1351...
Calling fib1350...
Calling fib1349...
Calling fib1348...
Calling fib1347...
Calling fib1346...
Calling fib1345...
Calling fib1344...
Calling fib1343...
Calling fib1342...
Calling fib1341...
Calling fib1340...
Calling fib1339...
Calling fib1338...
Calling fib1337...
Calling fib1336...
Calling fib1335...
Calling fib1334...
Calling fib1333...
Calling fib1332...
Calling fib1331...
Calling fib1330...

Calling fib1329...
Calling fib1328...
Calling fib1327...
Calling fib1326...
Calling fib1325...
Calling fib1324...
Calling fib1323...
Calling fib1322...
Calling fib1321...
Calling fib1320...
Calling fib1319...
Calling fib1318...
Calling fib1317...
Calling fib1316...
Calling fib1315...
Calling fib1314...
Calling fib1313...
Calling fib1312...
Calling fib1311...
Calling fib1310...
Calling fib1309...
Calling fib1308...
Calling fib1307...
Calling fib1306...
Calling fib1305...
Calling fib1304...
Calling fib1303...
Calling fib1302...
Calling fib1301...
Calling fib1300...
Calling fib1299...
Calling fib1298...
Calling fib1297...
Calling fib1296...
Calling fib1295...
Calling fib1294...
Calling fib1293...
Calling fib1292...
Calling fib1291...
Calling fib1290...
Calling fib1289...
Calling fib1288...
Calling fib1287...
Calling fib1286...
Calling fib1285...
Calling fib1284...
Calling fib1283...
Calling fib1282...
Calling fib1281...
Calling fib1280...
Calling fib1279...
Calling fib1278...
Calling fib1277...
Calling fib1276...
Calling fib1275...
Calling fib1274...
Calling fib1273...
Calling fib1272...
Calling fib1271...
Calling fib1270...
Calling fib1269...

Calling fib1268...
Calling fib1267...
Calling fib1266...
Calling fib1265...
Calling fib1264...
Calling fib1263...
Calling fib1262...
Calling fib1261...
Calling fib1260...
Calling fib1259...
Calling fib1258...
Calling fib1257...
Calling fib1256...
Calling fib1255...
Calling fib1254...
Calling fib1253...
Calling fib1252...
Calling fib1251...
Calling fib1250...
Calling fib1249...
Calling fib1248...
Calling fib1247...
Calling fib1246...
Calling fib1245...
Calling fib1244...
Calling fib1243...
Calling fib1242...
Calling fib1241...
Calling fib1240...
Calling fib1239...
Calling fib1238...
Calling fib1237...
Calling fib1236...
Calling fib1235...
Calling fib1234...
Calling fib1233...
Calling fib1232...
Calling fib1231...
Calling fib1230...
Calling fib1229...
Calling fib1228...
Calling fib1227...
Calling fib1226...
Calling fib1225...
Calling fib1224...
Calling fib1223...
Calling fib1222...
Calling fib1221...
Calling fib1220...
Calling fib1219...
Calling fib1218...
Calling fib1217...
Calling fib1216...
Calling fib1215...
Calling fib1214...
Calling fib1213...
Calling fib1212...
Calling fib1211...
Calling fib1210...
Calling fib1209...
Calling fib1208...

Calling fib1207...
Calling fib1206...
Calling fib1205...
Calling fib1204...
Calling fib1203...
Calling fib1202...
Calling fib1201...
Calling fib1200...
Calling fib1199...
Calling fib1198...
Calling fib1197...
Calling fib1196...
Calling fib1195...
Calling fib1194...
Calling fib1193...
Calling fib1192...
Calling fib1191...
Calling fib1190...
Calling fib1189...
Calling fib1188...
Calling fib1187...
Calling fib1186...
Calling fib1185...
Calling fib1184...
Calling fib1183...
Calling fib1182...
Calling fib1181...
Calling fib1180...
Calling fib1179...
Calling fib1178...
Calling fib1177...
Calling fib1176...
Calling fib1175...
Calling fib1174...
Calling fib1173...
Calling fib1172...
Calling fib1171...
Calling fib1170...
Calling fib1169...
Calling fib1168...
Calling fib1167...
Calling fib1166...
Calling fib1165...
Calling fib1164...
Calling fib1163...
Calling fib1162...
Calling fib1161...
Calling fib1160...
Calling fib1159...
Calling fib1158...
Calling fib1157...
Calling fib1156...
Calling fib1155...
Calling fib1154...
Calling fib1153...
Calling fib1152...
Calling fib1151...
Calling fib1150...
Calling fib1149...
Calling fib1148...
Calling fib1147...

Calling fib1146...
Calling fib1145...
Calling fib1144...
Calling fib1143...
Calling fib1142...
Calling fib1141...
Calling fib1140...
Calling fib1139...
Calling fib1138...
Calling fib1137...
Calling fib1136...
Calling fib1135...
Calling fib1134...
Calling fib1133...
Calling fib1132...
Calling fib1131...
Calling fib1130...
Calling fib1129...
Calling fib1128...
Calling fib1127...
Calling fib1126...
Calling fib1125...
Calling fib1124...
Calling fib1123...
Calling fib1122...
Calling fib1121...
Calling fib1120...
Calling fib1119...
Calling fib1118...
Calling fib1117...
Calling fib1116...
Calling fib1115...
Calling fib1114...
Calling fib1113...
Calling fib1112...
Calling fib1111...
Calling fib1110...
Calling fib1109...
Calling fib1108...
Calling fib1107...
Calling fib1106...
Calling fib1105...
Calling fib1104...
Calling fib1103...
Calling fib1102...
Calling fib1101...
Calling fib1100...
Calling fib1099...
Calling fib1098...
Calling fib1097...
Calling fib1096...
Calling fib1095...
Calling fib1094...
Calling fib1093...
Calling fib1092...
Calling fib1091...
Calling fib1090...
Calling fib1089...
Calling fib1088...
Calling fib1087...
Calling fib1086...

Calling fib1085...
Calling fib1084...
Calling fib1083...
Calling fib1082...
Calling fib1081...
Calling fib1080...
Calling fib1079...
Calling fib1078...
Calling fib1077...
Calling fib1076...
Calling fib1075...
Calling fib1074...
Calling fib1073...
Calling fib1072...
Calling fib1071...
Calling fib1070...
Calling fib1069...
Calling fib1068...
Calling fib1067...
Calling fib1066...
Calling fib1065...
Calling fib1064...
Calling fib1063...
Calling fib1062...
Calling fib1061...
Calling fib1060...
Calling fib1059...
Calling fib1058...
Calling fib1057...
Calling fib1056...
Calling fib1055...
Calling fib1054...
Calling fib1053...
Calling fib1052...
Calling fib1051...
Calling fib1050...
Calling fib1049...
Calling fib1048...
Calling fib1047...
Calling fib1046...
Calling fib1045...
Calling fib1044...
Calling fib1043...
Calling fib1042...
Calling fib1041...
Calling fib1040...
Calling fib1039...
Calling fib1038...
Calling fib1037...
Calling fib1036...
Calling fib1035...
Calling fib1034...
Calling fib1033...
Calling fib1032...
Calling fib1031...
Calling fib1030...
Calling fib1029...
Calling fib1028...
Calling fib1027...
Calling fib1026...
Calling fib1025...

Calling fib1024...
Calling fib1023...
Calling fib1022...
Calling fib1021...
Calling fib1020...
Calling fib1019...
Calling fib1018...
Calling fib1017...
Calling fib1016...
Calling fib1015...
Calling fib1014...
Calling fib1013...
Calling fib1012...
Calling fib1011...
Calling fib1010...
Calling fib1009...
Calling fib1008...
Calling fib1007...
Calling fib1006...
Calling fib1005...
Calling fib1004...
Calling fib1003...
Calling fib1002...
Calling fib1001...
Calling fib1000...
Calling fib999...
Calling fib998...
Calling fib997...
Calling fib996...
Calling fib995...
Calling fib994...
Calling fib993...
Calling fib992...
Calling fib991...
Calling fib990...
Calling fib989...
Calling fib988...
Calling fib987...
Calling fib986...
Calling fib985...
Calling fib984...
Calling fib983...
Calling fib982...
Calling fib981...
Calling fib980...
Calling fib979...
Calling fib978...
Calling fib977...
Calling fib976...
Calling fib975...
Calling fib974...
Calling fib973...
Calling fib972...
Calling fib971...
Calling fib970...
Calling fib969...
Calling fib968...
Calling fib967...
Calling fib966...
Calling fib965...
Calling fib964...

Calling fib963...
Calling fib962...
Calling fib961...
Calling fib960...
Calling fib959...
Calling fib958...
Calling fib957...
Calling fib956...
Calling fib955...
Calling fib954...
Calling fib953...
Calling fib952...
Calling fib951...
Calling fib950...
Calling fib949...
Calling fib948...
Calling fib947...
Calling fib946...
Calling fib945...
Calling fib944...
Calling fib943...
Calling fib942...
Calling fib941...
Calling fib940...
Calling fib939...
Calling fib938...
Calling fib937...
Calling fib936...
Calling fib935...
Calling fib934...
Calling fib933...
Calling fib932...
Calling fib931...
Calling fib930...
Calling fib929...
Calling fib928...
Calling fib927...
Calling fib926...
Calling fib925...
Calling fib924...
Calling fib923...
Calling fib922...
Calling fib921...
Calling fib920...
Calling fib919...
Calling fib918...
Calling fib917...
Calling fib916...
Calling fib915...
Calling fib914...
Calling fib913...
Calling fib912...
Calling fib911...
Calling fib910...
Calling fib909...
Calling fib908...
Calling fib907...
Calling fib906...
Calling fib905...
Calling fib904...
Calling fib903...

Calling fib902...
Calling fib901...
Calling fib900...
Calling fib899...
Calling fib898...
Calling fib897...
Calling fib896...
Calling fib895...
Calling fib894...
Calling fib893...
Calling fib892...
Calling fib891...
Calling fib890...
Calling fib889...
Calling fib888...
Calling fib887...
Calling fib886...
Calling fib885...
Calling fib884...
Calling fib883...
Calling fib882...
Calling fib881...
Calling fib880...
Calling fib879...
Calling fib878...
Calling fib877...
Calling fib876...
Calling fib875...
Calling fib874...
Calling fib873...
Calling fib872...
Calling fib871...
Calling fib870...
Calling fib869...
Calling fib868...
Calling fib867...
Calling fib866...
Calling fib865...
Calling fib864...
Calling fib863...
Calling fib862...
Calling fib861...
Calling fib860...
Calling fib859...
Calling fib858...
Calling fib857...
Calling fib856...
Calling fib855...
Calling fib854...
Calling fib853...
Calling fib852...
Calling fib851...
Calling fib850...
Calling fib849...
Calling fib848...
Calling fib847...
Calling fib846...
Calling fib845...
Calling fib844...
Calling fib843...
Calling fib842...

Calling fib841...
Calling fib840...
Calling fib839...
Calling fib838...
Calling fib837...
Calling fib836...
Calling fib835...
Calling fib834...
Calling fib833...
Calling fib832...
Calling fib831...
Calling fib830...
Calling fib829...
Calling fib828...
Calling fib827...
Calling fib826...
Calling fib825...
Calling fib824...
Calling fib823...
Calling fib822...
Calling fib821...
Calling fib820...
Calling fib819...
Calling fib818...
Calling fib817...
Calling fib816...
Calling fib815...
Calling fib814...
Calling fib813...
Calling fib812...
Calling fib811...
Calling fib810...
Calling fib809...
Calling fib808...
Calling fib807...
Calling fib806...
Calling fib805...
Calling fib804...
Calling fib803...
Calling fib802...
Calling fib801...
Calling fib800...
Calling fib799...
Calling fib798...
Calling fib797...
Calling fib796...
Calling fib795...
Calling fib794...
Calling fib793...
Calling fib792...
Calling fib791...
Calling fib790...
Calling fib789...
Calling fib788...
Calling fib787...
Calling fib786...
Calling fib785...
Calling fib784...
Calling fib783...
Calling fib782...
Calling fib781...

Calling fib780...
Calling fib779...
Calling fib778...
Calling fib777...
Calling fib776...
Calling fib775...
Calling fib774...
Calling fib773...
Calling fib772...
Calling fib771...
Calling fib770...
Calling fib769...
Calling fib768...
Calling fib767...
Calling fib766...
Calling fib765...
Calling fib764...
Calling fib763...
Calling fib762...
Calling fib761...
Calling fib760...
Calling fib759...
Calling fib758...
Calling fib757...
Calling fib756...
Calling fib755...
Calling fib754...
Calling fib753...
Calling fib752...
Calling fib751...
Calling fib750...
Calling fib749...
Calling fib748...
Calling fib747...
Calling fib746...
Calling fib745...
Calling fib744...
Calling fib743...
Calling fib742...
Calling fib741...
Calling fib740...
Calling fib739...
Calling fib738...
Calling fib737...
Calling fib736...
Calling fib735...
Calling fib734...
Calling fib733...
Calling fib732...
Calling fib731...
Calling fib730...
Calling fib729...
Calling fib728...
Calling fib727...
Calling fib726...
Calling fib725...
Calling fib724...
Calling fib723...
Calling fib722...
Calling fib721...
Calling fib720...

Calling fib719...
Calling fib718...
Calling fib717...
Calling fib716...
Calling fib715...
Calling fib714...
Calling fib713...
Calling fib712...
Calling fib711...
Calling fib710...
Calling fib709...
Calling fib708...
Calling fib707...
Calling fib706...
Calling fib705...
Calling fib704...
Calling fib703...
Calling fib702...
Calling fib701...
Calling fib700...
Calling fib699...
Calling fib698...
Calling fib697...
Calling fib696...
Calling fib695...
Calling fib694...
Calling fib693...
Calling fib692...
Calling fib691...
Calling fib690...
Calling fib689...
Calling fib688...
Calling fib687...
Calling fib686...
Calling fib685...
Calling fib684...
Calling fib683...
Calling fib682...
Calling fib681...
Calling fib680...
Calling fib679...
Calling fib678...
Calling fib677...
Calling fib676...
Calling fib675...
Calling fib674...
Calling fib673...
Calling fib672...
Calling fib671...
Calling fib670...
Calling fib669...
Calling fib668...
Calling fib667...
Calling fib666...
Calling fib665...
Calling fib664...
Calling fib663...
Calling fib662...
Calling fib661...
Calling fib660...
Calling fib659...

Calling fib658...
Calling fib657...
Calling fib656...
Calling fib655...
Calling fib654...
Calling fib653...
Calling fib652...
Calling fib651...
Calling fib650...
Calling fib649...
Calling fib648...
Calling fib647...
Calling fib646...
Calling fib645...
Calling fib644...
Calling fib643...
Calling fib642...
Calling fib641...
Calling fib640...
Calling fib639...
Calling fib638...
Calling fib637...
Calling fib636...
Calling fib635...
Calling fib634...
Calling fib633...
Calling fib632...
Calling fib631...
Calling fib630...
Calling fib629...
Calling fib628...
Calling fib627...
Calling fib626...
Calling fib625...
Calling fib624...
Calling fib623...
Calling fib622...
Calling fib621...
Calling fib620...
Calling fib619...
Calling fib618...
Calling fib617...
Calling fib616...
Calling fib615...
Calling fib614...
Calling fib613...
Calling fib612...
Calling fib611...
Calling fib610...
Calling fib609...
Calling fib608...
Calling fib607...
Calling fib606...
Calling fib605...
Calling fib604...
Calling fib603...
Calling fib602...
Calling fib601...
Calling fib600...
Calling fib599...
Calling fib598...

Calling fib597...
Calling fib596...
Calling fib595...
Calling fib594...
Calling fib593...
Calling fib592...
Calling fib591...
Calling fib590...
Calling fib589...
Calling fib588...
Calling fib587...
Calling fib586...
Calling fib585...
Calling fib584...
Calling fib583...
Calling fib582...
Calling fib581...
Calling fib580...
Calling fib579...
Calling fib578...
Calling fib577...
Calling fib576...
Calling fib575...
Calling fib574...
Calling fib573...
Calling fib572...
Calling fib571...
Calling fib570...
Calling fib569...
Calling fib568...
Calling fib567...
Calling fib566...
Calling fib565...
Calling fib564...
Calling fib563...
Calling fib562...
Calling fib561...
Calling fib560...
Calling fib559...
Calling fib558...
Calling fib557...
Calling fib556...
Calling fib555...
Calling fib554...
Calling fib553...
Calling fib552...
Calling fib551...
Calling fib550...
Calling fib549...
Calling fib548...
Calling fib547...
Calling fib546...
Calling fib545...
Calling fib544...
Calling fib543...
Calling fib542...
Calling fib541...
Calling fib540...
Calling fib539...
Calling fib538...
Calling fib537...

Calling fib536...
Calling fib535...
Calling fib534...
Calling fib533...
Calling fib532...
Calling fib531...
Calling fib530...
Calling fib529...
Calling fib528...
Calling fib527...
Calling fib526...
Calling fib525...
Calling fib524...
Calling fib523...
Calling fib522...
Calling fib521...
Calling fib520...
Calling fib519...
Calling fib518...
Calling fib517...
Calling fib516...
Calling fib515...
Calling fib514...
Calling fib513...
Calling fib512...
Calling fib511...
Calling fib510...
Calling fib509...
Calling fib508...
Calling fib507...
Calling fib506...
Calling fib505...
Calling fib504...
Calling fib503...
Calling fib502...
Calling fib501...
Calling fib500...
Calling fib499...
Calling fib498...
Calling fib497...
Calling fib496...
Calling fib495...
Calling fib494...
Calling fib493...
Calling fib492...
Calling fib491...
Calling fib490...
Calling fib489...
Calling fib488...
Calling fib487...
Calling fib486...
Calling fib485...
Calling fib484...
Calling fib483...
Calling fib482...
Calling fib481...
Calling fib480...
Calling fib479...
Calling fib478...
Calling fib477...
Calling fib476...

Calling fib475...
Calling fib474...
Calling fib473...
Calling fib472...
Calling fib471...
Calling fib470...
Calling fib469...
Calling fib468...
Calling fib467...
Calling fib466...
Calling fib465...
Calling fib464...
Calling fib463...
Calling fib462...
Calling fib461...
Calling fib460...
Calling fib459...
Calling fib458...
Calling fib457...
Calling fib456...
Calling fib455...
Calling fib454...
Calling fib453...
Calling fib452...
Calling fib451...
Calling fib450...
Calling fib449...
Calling fib448...
Calling fib447...
Calling fib446...
Calling fib445...
Calling fib444...
Calling fib443...
Calling fib442...
Calling fib441...
Calling fib440...
Calling fib439...
Calling fib438...
Calling fib437...
Calling fib436...
Calling fib435...
Calling fib434...
Calling fib433...
Calling fib432...
Calling fib431...
Calling fib430...
Calling fib429...
Calling fib428...
Calling fib427...
Calling fib426...
Calling fib425...
Calling fib424...
Calling fib423...
Calling fib422...
Calling fib421...
Calling fib420...
Calling fib419...
Calling fib418...
Calling fib417...
Calling fib416...
Calling fib415...

Calling fib414...
Calling fib413...
Calling fib412...
Calling fib411...
Calling fib410...
Calling fib409...
Calling fib408...
Calling fib407...
Calling fib406...
Calling fib405...
Calling fib404...
Calling fib403...
Calling fib402...
Calling fib401...
Calling fib400...
Calling fib399...
Calling fib398...
Calling fib397...
Calling fib396...
Calling fib395...
Calling fib394...
Calling fib393...
Calling fib392...
Calling fib391...
Calling fib390...
Calling fib389...
Calling fib388...
Calling fib387...
Calling fib386...
Calling fib385...
Calling fib384...
Calling fib383...
Calling fib382...
Calling fib381...
Calling fib380...
Calling fib379...
Calling fib378...
Calling fib377...
Calling fib376...
Calling fib375...
Calling fib374...
Calling fib373...
Calling fib372...
Calling fib371...
Calling fib370...
Calling fib369...
Calling fib368...
Calling fib367...
Calling fib366...
Calling fib365...
Calling fib364...
Calling fib363...
Calling fib362...
Calling fib361...
Calling fib360...
Calling fib359...
Calling fib358...
Calling fib357...
Calling fib356...
Calling fib355...
Calling fib354...

Calling fib353...
Calling fib352...
Calling fib351...
Calling fib350...
Calling fib349...
Calling fib348...
Calling fib347...
Calling fib346...
Calling fib345...
Calling fib344...
Calling fib343...
Calling fib342...
Calling fib341...
Calling fib340...
Calling fib339...
Calling fib338...
Calling fib337...
Calling fib336...
Calling fib335...
Calling fib334...
Calling fib333...
Calling fib332...
Calling fib331...
Calling fib330...
Calling fib329...
Calling fib328...
Calling fib327...
Calling fib326...
Calling fib325...
Calling fib324...
Calling fib323...
Calling fib322...
Calling fib321...
Calling fib320...
Calling fib319...
Calling fib318...
Calling fib317...
Calling fib316...
Calling fib315...
Calling fib314...
Calling fib313...
Calling fib312...
Calling fib311...
Calling fib310...
Calling fib309...
Calling fib308...
Calling fib307...
Calling fib306...
Calling fib305...
Calling fib304...
Calling fib303...
Calling fib302...
Calling fib301...
Calling fib300...
Calling fib299...
Calling fib298...
Calling fib297...
Calling fib296...
Calling fib295...
Calling fib294...
Calling fib293...

Calling fib292...
Calling fib291...
Calling fib290...
Calling fib289...
Calling fib288...
Calling fib287...
Calling fib286...
Calling fib285...
Calling fib284...
Calling fib283...
Calling fib282...
Calling fib281...
Calling fib280...
Calling fib279...
Calling fib278...
Calling fib277...
Calling fib276...
Calling fib275...
Calling fib274...
Calling fib273...
Calling fib272...
Calling fib271...
Calling fib270...
Calling fib269...
Calling fib268...
Calling fib267...
Calling fib266...
Calling fib265...
Calling fib264...
Calling fib263...
Calling fib262...
Calling fib261...
Calling fib260...
Calling fib259...
Calling fib258...
Calling fib257...
Calling fib256...
Calling fib255...
Calling fib254...
Calling fib253...
Calling fib252...
Calling fib251...
Calling fib250...
Calling fib249...
Calling fib248...
Calling fib247...
Calling fib246...
Calling fib245...
Calling fib244...
Calling fib243...
Calling fib242...
Calling fib241...
Calling fib240...
Calling fib239...
Calling fib238...
Calling fib237...
Calling fib236...
Calling fib235...
Calling fib234...
Calling fib233...
Calling fib232...

Calling fib231...
Calling fib230...
Calling fib229...
Calling fib228...
Calling fib227...
Calling fib226...
Calling fib225...
Calling fib224...
Calling fib223...
Calling fib222...
Calling fib221...
Calling fib220...
Calling fib219...
Calling fib218...
Calling fib217...
Calling fib216...
Calling fib215...
Calling fib214...
Calling fib213...
Calling fib212...
Calling fib211...
Calling fib210...
Calling fib209...
Calling fib208...
Calling fib207...
Calling fib206...
Calling fib205...
Calling fib204...
Calling fib203...
Calling fib202...
Calling fib201...
Calling fib200...
Calling fib199...
Calling fib198...
Calling fib197...
Calling fib196...
Calling fib195...
Calling fib194...
Calling fib193...
Calling fib192...
Calling fib191...
Calling fib190...
Calling fib189...
Calling fib188...
Calling fib187...
Calling fib186...
Calling fib185...
Calling fib184...
Calling fib183...
Calling fib182...
Calling fib181...
Calling fib180...
Calling fib179...
Calling fib178...
Calling fib177...
Calling fib176...
Calling fib175...
Calling fib174...
Calling fib173...
Calling fib172...
Calling fib171...

Calling fib170...
Calling fib169...
Calling fib168...
Calling fib167...
Calling fib166...
Calling fib165...
Calling fib164...
Calling fib163...
Calling fib162...
Calling fib161...
Calling fib160...
Calling fib159...
Calling fib158...
Calling fib157...
Calling fib156...
Calling fib155...
Calling fib154...
Calling fib153...
Calling fib152...
Calling fib151...
Calling fib150...
Calling fib149...
Calling fib148...
Calling fib147...
Calling fib146...
Calling fib145...
Calling fib144...
Calling fib143...
Calling fib142...
Calling fib141...
Calling fib140...
Calling fib139...
Calling fib138...
Calling fib137...
Calling fib136...
Calling fib135...
Calling fib134...
Calling fib133...
Calling fib132...
Calling fib131...
Calling fib130...
Calling fib129...
Calling fib128...
Calling fib127...
Calling fib126...
Calling fib125...
Calling fib124...
Calling fib123...
Calling fib122...
Calling fib121...
Calling fib120...
Calling fib119...
Calling fib118...
Calling fib117...
Calling fib116...
Calling fib115...
Calling fib114...
Calling fib113...
Calling fib112...
Calling fib111...
Calling fib110...

Calling fib109...
Calling fib108...
Calling fib107...
Calling fib106...
Calling fib105...
Calling fib104...
Calling fib103...
Calling fib102...
Calling fib101...
Calling fib100...
Calling fib99...
Calling fib98...
Calling fib97...
Calling fib96...
Calling fib95...
Calling fib94...
Calling fib93...
Calling fib92...
Calling fib91...
Calling fib90...
Calling fib89...
Calling fib88...
Calling fib87...
Calling fib86...
Calling fib85...
Calling fib84...
Calling fib83...
Calling fib82...
Calling fib81...
Calling fib80...
Calling fib79...
Calling fib78...
Calling fib77...
Calling fib76...
Calling fib75...
Calling fib74...
Calling fib73...
Calling fib72...
Calling fib71...
Calling fib70...
Calling fib69...
Calling fib68...
Calling fib67...
Calling fib66...
Calling fib65...
Calling fib64...
Calling fib63...
Calling fib62...
Calling fib61...
Calling fib60...
Calling fib59...
Calling fib58...
Calling fib57...
Calling fib56...
Calling fib55...
Calling fib54...
Calling fib53...
Calling fib52...
Calling fib51...
Calling fib50...
Calling fib49...

```
Calling fib48...
Calling fib47...
Calling fib46...
Calling fib45...
Calling fib44...
Calling fib43...
Calling fib42...
Calling fib41...
Calling fib40...
Calling fib39...
Calling fib38...
Calling fib37...
Calling fib36...
Calling fib35...
Calling fib34...
Calling fib33...
Calling fib32...
Calling fib31...
Calling fib30...
Calling fib29...
Calling fib28...
Calling fib27...
Calling fib26...
Calling fib25...
Calling fib24...
Calling fib23...
Calling fib22...
Calling fib21...
Calling fib20...
Calling fib19...
Calling fib18...
Calling fib17...
Calling fib16...
Calling fib15...
Calling fib14...
Calling fib13...
Calling fib12...
Calling fib11...
Calling fib10...
Calling fib9...
Calling fib8...
Calling fib7...
Calling fib6...
Calling fib5...
Calling fib4...
Calling fib3...
Calling fib2...
Calling fib1...
Calling fib0...
4224696333392304878706725602341482782579852840250681098010280137314308584370130707224123599639141511
0884460875389096036076401947116435960292719833125987373262535558026069915859152294924539049987222567
9531698287448247299226390183371677806060701161549788671987985831146887087626459736908672288402365442
2295243347964480139515349562972087652656069529806499841977448720155612802665404554171717881930324025
204312082516817125
```

In [370]:

```
1 def myTimer(f):
2
3     def callF(*args):
4         start = time.perf_counter()
5         result = f(*args)
```

```

6         end = time.perf_counter()
7         print(toEnggNotation(end - start))
8         return result
9
10        return callF
11
12
13    # timedPower = myTimer(math.pow)
14    # print(timedPower(5, 188))
15    #
16    #
17    @myTimer
18    def myPower(a, b):
19        return a ** b
20
21
22    print(myPower(5, 188))

```

```

2.5650µ 2.5650000452515087e-06
2548947057811923643246208636428338888945767288308190057718217365417860642420893057508943799552292297
50557351508177816867828369140625

```

```
In [371]: 1 PRECISION = 4
```

```
In [372]: 1 a = decimal.Decimal(math.e)
2 print(a)
3 print(type(a))
4 print(float(a))
```

```

2.718281828459045090795598298427648842334747314453125
<class 'decimal.Decimal'>
2.718281828459045

```

```
In [373]: 1 a = decimal.Decimal("2.404040313893824388847472828")
2 print(a)
3 print(type(a))
4 print(float(a))
```

```

2.404040313893824388847472828
<class 'decimal.Decimal'>
2.4040403138938244

```

```
In [374]: 1 a = decimal.Decimal(2.2384892948592929275758693)
2 print(a)
3 print(type(a))
4 print(float(a))
```

```

2.23848929485929293292656439007259905338287353515625
<class 'decimal.Decimal'>
2.238489294859293

```

```
In [375]: 1 l = [decimal.Decimal(random.random()) for _ in range(10)]
2 a = sum(l) / len(l)
```

```
3 print(a)
4 print(type(a))
```

```
0.6355070068397963400919081777
<class 'decimal.Decimal'>
```

In [376]:

```
1 def normalize(f):
2
3     def callF(*args):
4         result = f(*args)
5         # print(type(result))
6         result = float(result)
7         return float(f"{result:.{PRECISION}f}")
8
9     return callF
```

In [377]:

```
1 @normalize
2 def perc_diff(x, y):
3     try:
4         return (y - x) / x * 100
5     except ZeroDivisionError:
6         return 0
7
8
9 @normalize
10 def sum_squares(*args):
11     return sum(x ** 2 for x in args)
12
13
14 @normalize
15 def avg(*args):
16     if len(args) == 0:
17         return 0
18
19     numbers = [decimal.Decimal(x) for x in args]
20     sum_ = sum(numbers)
21     return sum_ / len(numbers)
```

In [378]:

```
1 perc_diff(5088, 2872)
```

Out[378]: -43.5535

In [379]:

```
1 avg(2.4, 4.1, 5.932929, 8.2838485)
```

Out[379]: 5.1792

In [380]:

```
1 def add(x, y):
2     time.sleep(2)
3     return x + y
4
```

```
5 print(timeit.timeit("add(2340320832580385043843805805,  
6 320342408320482804320584305)", globals = globals(), number = 10))
```

20.000953659999993

In [381]:

```
1 @functools.lru_cache  
2 def add(x, y):  
3     time.sleep(2)  
4     return x + y  
5  
6  
7 print(timeit.timeit("add(2340320832580385043843805805,  
320342408320482804320584305)", globals = globals(), number = 10))
```

2.0001189919999547

In [382]:

```
1 def add(x, y):  
2     time.sleep(2)  
3     return x + y  
4  
5  
6 print(timeit.repeat("add(2340320832580385043843805805,  
320342408320482804320584305)", globals = globals(), number = 10))
```

[20.000912009000047, 20.000890174000006, 20.000836774999982, 20.00079347799999, 20.000815604000024]

In [383]:

```
1 @functools.lru_cache  
2 def add(x, y):  
3     time.sleep(2)  
4     return x + y  
5  
6  
7 print(timeit.repeat("add(2340320832580385043843805805,  
320342408320482804320584305)", globals = globals(), number = 10))
```

[2.000094929999989, 1.943999961895315e-06, 1.5530000041508174e-06, 1.4529999816659256e-06,
1.4830000054644188e-06]

In [384]:

```
1 def graham(x, y, z):  
2     return x ** y ** z  
3  
4  
5 print(toEnggNotation(timeit.timeit("graham(4, 5, 6)", globals = globals(),  
number = 10)))
```

686.5070μ 0.0006865070000117157

In [385]:

```
1 @functools.lru_cache  
2 def graham(x, y, z):  
3     return x ** y ** z  
4  
5  
6 print(toEnggNotation(timeit.timeit("graham(2, 3, 4)", globals = globals(),  
number = 10)))
```

3.9570μ 3.9569999898958486e-06

```
In [386]: 1 print(toEnggNotation(timeit.timeit("2 ** 3 ** 4", number = 100)))
```

18.8660μ 1.8866000004891248e-05

```
In [387]: 1 def normalize(precision):
2
3         def callF(f):
4
5             def callFInner(*args):
6                 result = f(*args)
7                 result = f"{result:.{precision}f}"
8                 return result
9
10            return callFInner
11
12        return callF
```

```
In [388]: 1 @normalize(2)
2 def perc_diff(x, y):
3     try:
4         return (y - x) / x * 100
5     except ZeroDivisionError:
6         return 0
7
8
9 @normalize(4)
10 def sum_squares(*args):
11     return sum(x ** 2 for x in args)
12
13
14 @normalize(8)
15 def avg(*args):
16     if len(args) == 0:
17         return 0
18
19     numbers = [decimal.Decimal(x) for x in args]
20     sum_ = sum(numbers)
21     return sum_ / len(numbers)
```

```
In [389]: 1 print(perc_diff(3, 7))
2 print(sum_squares(3, 4, 5))
3 print(avg(1.25, 1.75, 1.5, 1.0, 2.0, 2.5, 2.25, 2.75))
```

133.33
50.0000
1.87500000

```
In [390]: 1 def normalize(f):
2
3         def callF(*args):
```

```

4         result = f(*args)
5         return result
6
7     return callF

```

In [391]:

```

1  try:
2
3      @normalize(2)
4      def perc_diff(x, y):
5          try:
6              return (y - x) / x * 100
7          except ZeroDivisionError:
8              return 0
9
10     @normalize(4)
11     def sum_squares(*args):
12         return sum(x ** 2 for x in args)
13
14     @normalize(8)
15     def avg(*args):
16         if len(args) == 0:
17             return 0
18
19         numbers = [decimal.Decimal(x) for x in args]
20         sum_ = sum(numbers)
21         return sum_ / len(numbers)
22
23 except TypeError as ex:
24     print(type(ex).      + ": " + ex.args[0] + ".")

```

TypeError: 'int' object is not callable.

In [392]:

```

@functools.lru_cache
1 def getFibonacci(i):
2     if i == 0:
3         return 1
4     if i == 1:
5         return 1
6     if i < 0:
7         return 0
8     else:
9         return getFibonacci(i - 1) + getFibonacci(i - 2)
10
11 print(getFibonacci(60))
12
13 with open("../var/out000.txt", "w") as fl:
14     fl.write("i,fib,fact,gcd\n")
15     for i in range(101):
16         fl.write(f"{i},{math.factorial(i)},{getFibonacci(i)},
17 {math.gcd(getFibonacci(i),math.factorial(i))}\n")
18
19 0

```

```
1 with open("../var/out000.txt") as fl:
1   for line in fl:
1     print(line.strip())
2
1
3
1
4
1
5
1
6
1
7
1
8
1
9
2
0
2
1
2
2
```

2504730781961
i, fib, fact, gcd
0, 1, 1, 1
1, 1, 1, 1
2, 2, 2, 2
3, 6, 3, 3
4, 24, 5, 1
5, 120, 8, 8
6, 720, 13, 1
7, 5040, 21, 21
8, 40320, 34, 2
9, 362880, 55, 5
10, 3628800, 89, 1
11, 39916800, 144, 144
12, 479001600, 233, 1
13, 6227020800, 377, 13
14, 87178291200, 610, 10
15, 1307674368000, 987, 21
16, 20922789888000, 1597, 1
17, 355687428096000, 2584, 136
18, 6402373705728000, 4181, 1
19, 121645100408832000, 6765, 165
20, 2432902008176640000, 10946, 26
21, 51090942171709440000, 17711, 1
22, 112400072777607680000, 28657, 1
23, 25852016738884976640000, 46368, 46368
24, 620448401733239439360000, 75025, 25
25, 1551121004330985984000000, 121393, 1
26, 403291461126605635584000000, 196418, 34
27, 10888869450418352160768000000, 317811, 39
28, 304888344611713860501504000000, 514229, 1
29, 8841761993739701954543616000000, 832040, 440
30, 265252859812191058636308480000000, 1346269, 1
31, 8222838654177922817725562880000000, 2178309, 21
32, 263130836933693530167218012160000000, 3524578, 2
33, 8683317618811886495518194401280000000, 5702887, 1
34, 295232799039604140847618609643520000000, 9227465, 65
35, 10333147966386144929666651337523200000000, 14930352, 139536
36, 371993326789901217467999448150835200000000, 24157817, 1
37, 13763753091226345046315979581580902400000000, 39088169, 37
38, 523022617466601111760007224100074291200000000, 63245986, 2
39, 20397882081197443358640281739902897356800000000, 102334155, 1155
40, 815915283247897734345611269596115894272000000000, 165580141, 1
41, 33452526613163807108170062053440751665152000000000, 267914296, 3016
42, 1405006117752879898543142606244511569936384000000000, 433494437, 1
43, 60415263063373835637355132068513997507264512000000000, 701408733, 129
44, 2658271574788448768043625811014615890319638528000000000, 1134903170, 170
45, 119622220865480194561963161495657715064383733760000000000, 1836311903, 1
46, 5502622159812088949850305428800254892961651752960000000000, 2971215073, 1
47, 258623241511168180642964355153611979969197632389120000000000, 4807526976, 4358592
48, 12413915592536072670862289047373375038521486354677760000000000, 7778742049, 13
49, 608281864034267560872252163321295376887552831379210240000000000, 12586269025, 275
50, 30414093201713378043612608166064768844377641568960512000000000000, 20365011074, 2
51, 1551118753287382280224243016469303211063259720016986112000000000000, 32951280099, 3
52, 80658175170943878571660636856403766975289505440883277824000000000000, 53316291173, 1
53, 4274883284060025564298013753389399649690343788366813724672000000000000, 86267571272, 136952
54, 230843697339241380472092742683027581083278564571807941132288000000000000, 139583862445, 5
55, 1269640335365827592596510084756651695958032105144943676227584000000000000, 225851433717, 55419
56, 71099858780486345185404564746372494973649797888116845868744704000000000000, 365435296162, 74
57, 4052691950487721675568060190543232213498038479622660214518448128000000000000, 591286729879, 1
58, 235056133128287857182947491051507468382886231818114292442069991424000000000000, 956722026041, 1

59,13868311854568983573793901972038940634590287677268743254082129494016000000000000,1548008755920,1
0066320
60,832098711274139014427634118322336438075417260636124595244927769640960000000000000,2504730781961,
1
61,50758021387722479880085681217662522722600452898803600309940593948098560000000000000,405273953788
1,1
62,3146997326038793752565312235495076408801228079725823219216316824782110720000000000000,6557470319
842,442
63,198260831540444006411614670836189813754477369022726862810627959961272975360000000000000,10610209
857723,987
64,12688693218588416410343338933516148080286551617454519219880189437521470423040000000000000,171676
80177565,5
65,824765059208247066672317030678549625218625855134543749292212313438895577497600000000000000,27777
890035288,8
66,54434493907744306400372924024784275264429306438879887453286012686967108114841600000000000000,449
45570212853,1
67,3647111091818868528824985909660546442716763531404952459370162850026796243694387200000000000000,7
2723460248141,201
68,248003554243683059960099041856917158104739920135536767237171073801822144571218329600000000000000
,117669030460994,2
69,1711224524281413113724683388812728390922705448935203693936480409232572797541406474240000000000000
00,190392490709135,20735
70,119785716699698917960727837216890987364589381425464258575536286462800958278984531968000000000000
0000,308061521170129,1
71,8504785885678623175211676442399260102885846081207962358864307633885886803780790176972800000000000
00000,498454011879264,44930592
72,6123445837688608686152407038527467274077809178469732898382301496397838498722168927420416000000000
0000000,806515533049393,1
73,44701154615126843408912571381250511100768007002829050158190800923704221040671833170169036800000000
000000000,1304969544928657,73
74,3307885441519386412259530282212537821456832518209349711706119268354112357009715654592508723200000
00000000000,2111485077978050,3050
75,2480914081139539809194647711659403366092624388657012283779589451265584267757286740944381542400000
000000000000,3416454622906707,111
76,1885494701666050254987932260861146558230394535379329335672487982961844043495537923117729972224000
0000000000000,5527939700884757,13
77,1451830920282858696340707840863082849837403792242083588467815746880619913491564200800652078612480
0000000000000000,8944394323791464,8
78,1132428117820629783145752115873204622873174957948825199004896282566883532523420076624508621317734
400000000000000000,14472334024676221,1
79,8946182130782975286851441715398316520698082167795719072138680632278379906935018605333618108410101
7600000000000000000,23416728348467685,2225685
80,7156945704626380229481153372318653216558465734236575257710944505822703925548014884266894486728081
408000000000000000000,37889062373143906,1802
81,5797126020747367985879734231578109105412357244731625958745865049716390179693892056256184534249745
94048000000000000000000,61305790721611591,1
82,4753643337012841748421382069894049466438132940679933286171609340767439947348991486130071318084791
6711936000000000000000000,99194853094755497,1
83,3945523969720658651189747118012061057143650340764344627522435752836975156299662933487959194010377
087090688000000000000000000,160500643816367088,4505904
84,3314240134565353266999387579130131288000666286242049487118846032383059131291716864129885722968716
7531561779200000000000000000,259695496911122585,5
85,2817104114380550276949479442260611594800566343305742064051019127525600261597959334510402864523409
240182751232000000000000000000,420196140727489673,1
86,2422709538367273238176552320344125971528487055242938175083876449672016224974245027678946463490131
94655716605952000000000000000000,679891637638612258,2
87,2107757298379527717213600518699389595229783738061356212322972511214654115727593174080683423236414
7935047344717824000000000000000000,1100087778366101931,903
88,1854826422573984391147968456455462843802209689493993466844215809868895621840281993191001412448045
018284166335168512000000000000000000,1779979416004714189,1
89,1650795516090846108121691926245361930983966623649654185491352070783317103437850973939991257078760

```

066272908038299975680000000000000000000,2880067194370816120,268748920
90,1485715964481761497309522733620825737885569961284688766942216863704985393094065876545992131370884
05964561723446997811200000000000000000000,4660046610375530309,169
91,1352001527678402962551665687594951421475868664769066777917417345971536707715599947656852839547504
4942775116833676800819200000000000000000000,7540113804746346429,3
92,1243841405464130725547532432587355307757799171587541435684023958293813771098351951844304612383704
134735310748698265675366400000000000000000000,12200160415121876738,2
93,1156772507081641574759205162306240436214753229576413535186142281213246807121467315215203289516844
84530383899628938707809075200000000000000000000,19740274219868223167,1
94,1087366156656743080273652852567866010041868035801828723074973744340451998694179276302291092145834
1545856086565120238534053068800000000000000000000,31940434634990099905,185
95,1032997848823905926259970209939472709539774634011737286921225057123429398759470312487176537538542
44685632822368642266073504153600000000000000000000,51680708854858323072,8717184
96,9916779348709496892095714015418938011581836486512677954443760548384922228090914999876894760370007
489820750947389657543056398745600000000000000000000,83621143489848422977,1
97,9619275968248211985332842594956369871234381391917297615810447731933374561248187549880587917558907
26512612841896796781676470678323200000000000000000000,135301852344706746049,36569
98,942689044888324774562618574305724247380969376407895166349423877294707070023223798882976159207729
1198236058505886084604294126475673600000000000000000000,218922995834555169026,3026
99,9332621544394415268169923885626670049071596826438162146859296389521759999322991560894146397615651
828625369792082722375825118521091686400000000000000000000,354224848179261915075,33825
100,933262154439441526816992388562667004907159682643816214685929638952175999932299156089414639761565
1828625369792082722375825118521091686400000000000000000000,573147844013817084101,1

```

In [393]:

```

fibonacciNumbers = {}
1 factorialNumbers = {}
gcd = {}
2
with open("../var/out000.txt") as fl:
3     next(fl)
    for ln in fl:
4         l = ln.strip().split(",")
        factorialNumbers[int(l[0])] = int(l[1])
5         fibonacciNumbers[int(l[0])] = int(l[2])
        gcd[(int(l[1]), int(l[2]))] = int(l[3])
6
7 # @functools.lru_cache
def getFibonacci(i):
8     print(f"getFibonacci({i}):")
    if i < 0:
9         return 0
    if i in fibonacciNumbers:
1        print("Found in cache!")
        return fibonacciNumbers[i]
1        print("Not found in cache...")
    fibonacciNumbers[i] = getFibonacci(i - 1) + getFibonacci(i - 2)
2        return fibonacciNumbers[i]
1
3
1 print(getFibonacci(110))
4 print(getFibonacci(11))
1
5
1 def f(i):

```

```

6
1      def g():
7          return math.factorial(i)
1
8      return g
1
9
2  def getFactorial(i):
0      print(f"getFactorial({i}):")
2      if i in factorialNumbers:
1          print("Found in cache!")
2          return factorialNumbers[i]
2      print("Not found in cache...")
2      return f(i)()
3          # return factorialNumbers.get(i, f(i)())      # Doesn't work because
2      the closure is always called. Have to use condition.
4
2
5  print(getFactorial(110))
2  print(getFactorial(11))
6
2
7  def f1(i, j):
2
8      def g1():
2          return math.gcd(i, j)
9
3      return g1
0
3
1  def getGCD(i, j):
3      print(f"getGCD({i, j}):")
2      if (i, j) in gcd:
3          print("Found in cache!")
3          return gcd[(i, j)]
3      print("Not found in cache...")
4      return math.gcd(i, j)
3
5
3  print(getGCD(5040, 21))
6  print(getGCD(5040, 22))
3
7
3
8
3
9
4
0
4
1
4

```

2
4
3
4
4
4
5
4
6
4
7
4
8
4
9
5
0
5
1
5
2
5
3
5
4
5
5
5
6
5
7
5
8
5
9
6
0
6
1
6
2
6
3
6
4
6
5
6
6
6
7
6

8
6
9
7
0
7
1

[illegible]

```
In [394]: 1 def importMPL():
2         import matplotlib as mpl
3
4         return mpl
5
6
7 mpl = importMPL()
8 print(dir(mpl))
```

```
['ExecutableNotFoundError', 'MatplotlibDeprecationWarning', 'MutableMapping', 'Parameter', 'Path', 'RcParams', '_ExecInfo', '_VersionInfo', '__all__', '__bibtex__', '__builtins__', '__cached__', '__doc__', '__file__', '__getattr__', '__loader__', '__name__', '__package__', '__path__', '__spec__', '_add_data_doc', '_api', '_c_internal_utils', '_check_versions', '_cm', '_cm_bivar', '_cm_listed', '_cm_multivar', '_color_data', '_deprecated_ignore_map', '_deprecated_map', '_deprecated_remain_as_none', '_docstring', '_ensure_handler', '_enums', '_fontconfig_pattern', '_get_config_or_cache_dir', '_get_executable_info', '_get_ssl_context', '_get_version', '_get_xdg_cache_dir', '_get_xdg_config_dir', '_image', '_init_tests', '_label_from_arg', '_log', '_logged_cached', '_open_file_or_url', '_parse_to_version_info', '_path', '_preprocess_data', '_rc_params_in_file', '_replacer', '_val_or_rc', '_version', 'artist', 'atexit', 'backends', 'bezier', 'bivar_colormaps', 'cbook', 'cm', 'color_sequences', 'colorizer', 'colormaps', 'colors', 'contextlib', 'cycler', 'defaultParams', 'ft2font', 'functools', 'get_backend', 'get_cachedir', 'get_configdir', 'get_data_path', 'importlib', 'inspect', 'interactive', 'is_interactive', 'locale', 'logging', 'matplotlib_fname', 'multivar_colormaps', 'namedtuple', 'os', 'parse_version', 'path', 'pprint', 'rc', 'rcParams', 'rcParamsDefault', 'rcParamsOrig', 'rc_context', 'rc_file', 'rc_file_defaults', 'rc_params', 'rc_params_from_file', 'rcdefaults', 'rcsetup', 're', 'sanitize_sequence', 'scale', 'set_loglevel', 'shutil', 'subprocess', 'sys', 'tempfile', 'ticker', 'transforms', 'use', 'validate_backend']
```

```
In [395]: 1 epochTime = time.gmtime(0)
2         print(epochTime.tm_year, epochTime.tm_mday, epochTime.tm_hour,
epochTime.tm_min, epochTime.tm_zone)
```

```
1970 1 0 0 GMT
```

```
In [396]: 1 time.perf_counter()
```

```
Out[396]: 440.340111055
```

```
In [397]: 1 start = time.perf_counter()
2         time.sleep(3)
3         end = time.perf_counter()
4         print(end - start)
```

```
3.0002507969999783
```

```
In [398]: 1 print(time.strftime("%I:%M %p %z", time.gmtime(time.time())))
```

```
12:00 PM +0000
```

```
In [ ]: 1 print(datetime.datetime.now(datetime.UTC))
2        print(datetime.datetime.now())
```

```
2026-09-18 12:00:08.720941+00:00
2026-09-18 17:30:08.721215
```

```
In [400]: 1 print(time.strftime("%Y-%m-%dT%H:%M:%S%z", time.gmtime(time.time())))
```

2026-09-18T12:00:08+0000

```
In [401]: my_time_struct = time.strptime("2020-09-17T16:36:16Z", "%Y-%m-%dT%H:%M:%S%z")
1 print(my_time_struct.tm_year, my_time_struct.tm_mon, my_time_struct.tm_mday,
2 my_time_struct.tm_hour, my_time_struct.tm_min, my_time_struct.tm_sec,
my_time_struct.tm_gmtoff)
```

2020 9 17 16 36 16 0

```
In [402]: my_time_struct = time.strptime("2020-09-17T16:36:16+0530", "%Y-%m-%dT%H:%M:%S%z")
1
2 print(my_time_struct.tm_year, my_time_struct.tm_mon, my_time_struct.tm_mday,
my_time_struct.tm_hour, my_time_struct.tm_min, my_time_struct.tm_sec,
my_time_struct.tm_gmtoff)
```

2020 9 17 16 36 16 19800

```
In [403]: 1 print("hello")
```

hello

```
In [404]: 1 s = "Monday, April 18, 2020"
2 myTimeStruct = time.strptime(s, "%A, %B %d, %Y")
3 print(myTimeStruct)
```

time.struct_time(tm_year=2020, tm_mon=4, tm_mday=18, tm_hour=0, tm_min=0, tm_sec=0, tm_wday=0, tm_yday=109, tm_isdst=-1)

```
In [420]: 1 myDate = datetime.date(2020, 8, 20)
2 print(myDate)
3 print(repr(myDate))
4 print(myDate.day, myDate.month)
```

2020-08-20
datetime.date(2020, 8, 20)
20 8

```
In [419]: 1 print(myDate.isoformat())
```

2020-08-20

```
In [421]: 1 myDate = datetime.date.today()
2 print(myDate)
3 print(repr(myDate))
4 print(myDate.day, myDate.month)
```

2026-09-18
datetime.date(2026, 9, 18)
18 9

```
In [ ]: 1 print(myDate.isoformat())
```

2026-09-18
18 9

```
In [407]: 1 myDate = datetime.date.fromtimestamp(time.time())
          2 print(myDate.year)
```

2026

```
In [408]: 1 myDate = datetime.date.fromtimestamp(0)
          2 print(myDate.year)
```

1970

```
In [ ]: 1 myDate = datetime.date.fromisoformat("2020-08-24")
        2 print(myDate)
        3 print(repr(myDate))
```

2020-08-24

datetime.date(2020, 8, 24)

```
In [410]: 1 myTime = datetime.time(15, 23, 45)
          2 print(myTime.isoformat())
```

15:23:45

```
In [411]: 1 print(datetime.time(15, 30, 45, 600))
```

15:30:45.000600

```
In [424]: 1 try:
          2     myTime = datetime.time.now()
          3     print(myTime)
          4 except AttributeError as ex:
          5     print(type(ex).__name__, ex.args)
```

AttributeError ("type object 'datetime.time' has no attribute 'now'",)

```
In [412]: 1 print(datetime.time.fromisoformat("12:34:45.123"))
```

12:34:45.123000

```
In [413]: 1 myTime = datetime.time.fromisoformat("12:34:45.1234")
          2 print(myTime)
          3 print(myTime.hour)
          4 print(myTime.microsecond)
          5 try:
          6     print(myTime[:3])
          7 except TypeError as ex:
          8     print(type(ex).__name__, ex.args)
```

12:34:45.123400

12

123400

TypeError ("'datetime.time' object is not subscriptable",)

```
In [423]: 1 try:
          2     myTime = datetime.time.fromtimestamp(0)
          3     print(myTime)
```

```
4 except AttributeError as ex:
5     print(type(ex)., ex.args)
```

AttributeError ("type object 'datetime.time' has no attribute 'fromtimestamp',)

```
In [414]: 1 myDateTime = datetime.datetime(2020, 12, 9, 12, 43, 32, 38829)
2         print(myDateTime.isoformat())
```

2020-12-09T12:43:32.038829

```
In [426]: 1 myDateTime = datetime.datetime.now()
2         print(myDateTime)
3         print(repr(myDateTime))
4         print(myDateTime.hour)
5         print(myDateTime.isoformat())
```

2026-09-18 18:03:08.224995
datetime.datetime(2026, 9, 18, 18, 3, 8, 224995)
18
2026-09-18T18:03:08.224995

```
In [429]: 1 myDateTime = datetime.datetime.fromtimestamp(0)
2         print(myDateTime)
3         print(myDateTime.hour)
4         print(myDateTime.isoformat())
```

1970-01-01 05:30:00
5
1970-01-01T05:30:00

```
In [436]: 1 myDateTime = datetime.datetime.fromtimestamp(time.time())
2         print(myDateTime)
3         print(myDateTime.hour)
4         print(myDateTime.isoformat())
```

2026-09-18 18:11:43.908956
18
2026-09-18T18:11:43.908956

```
In [431]: 1 myDateTime = datetime.datetime.fromisoformat("2025-02-02T15:23:44+0545")
2         print(myDateTime)
3         print(myDateTime.hour)
4         print(myDateTime.isoformat())
```

2025-02-02 15:23:44+05:45
15
2025-02-02T15:23:44+05:45

```
In [432]: 1 myDateTime = datetime.datetime.fromisoformat("2025-02-02T15:23:44")
2         print(myDateTime)
3         print(myDateTime.hour)
4         print(myDateTime.isoformat())
```

2025-02-02 15:23:44
15
2025-02-02T15:23:44

```
In [433]: 1 myDateTime = datetime.datetime.fromisoformat("2025-02-02 15:23:44")
          2 print(myDateTime)
          3 print(myDateTime.hour)
          4 print(myDateTime.isoformat())
```

```
2025-02-02 15:23:44
15
2025-02-02T15:23:44
```

```
In [434]: 1 myDateTime = datetime.datetime.fromisoformat("2025-02-02 15:23")
          2 print(myDateTime)
          3 print(myDateTime.hour)
          4 print(myDateTime.isoformat())
```

```
2025-02-02 15:23:00
15
2025-02-02T15:23:00
```

```
In [435]: 1 myDateTime = datetime.datetime.fromisoformat("2025-02-02")
          2 print(myDateTime)
          3 print(myDateTime.hour)
          4 print(myDateTime.isoformat())
```

```
2025-02-02 00:00:00
0
2025-02-02T00:00:00
```

```
In [438]: 1 myDateTime = datetime.datetime.utcnow()
          2 print(myDateTime)
          3 print(myDateTime.hour)
          4 print(myDateTime.tzinfo)
          5 print(myDateTime.isoformat())
```

```
2026-09-18 12:54:43.306385
12
None
2026-09-18T12:54:43.306385

/tmp/ipykernel_18021/573860576.py:1: DeprecationWarning: datetime.datetime.utcnow() is deprecated
and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in
UTC: datetime.datetime.now(datetime.UTC).
  myDateTime = datetime.datetime.utcnow()
```

```
In [440]: 1 myDateTime = datetime.datetime.now(datetime.UTC)
          2 print(myDateTime)
          3 print(myDateTime.hour)
          4 print(myDateTime.tzinfo)
          5 print(myDateTime.isoformat())
```

```
2026-09-18 12:55:53.446540+00:00
12
UTC
2026-09-18T12:55:53.446540+00:00
```

```
In [442]: 1 try:
          2     myDateTime = datetime.datetime.now(datetime.IST)
          3     print(myDateTime)
```

```

4     print(myDateTime.hour)
5     print(myDateTime.tzinfo)
6     print(myDateTime.isoformat())
7 except AttributeError as ex:
8     print(type(ex).      , ex.args)

```

AttributeError ("module 'datetime' has no attribute 'IST',)

In [415]: 1 print(cmath.sqrt(-4))

2j

In [451]:

```

1 dt1 = datetime.datetime.now()
2 print(dt1)
3 dt2 = datetime.datetime.today()
4 print(dt2)
5 td = dt1 - dt2
6 print(toEnggNotation(abs(td.total_seconds()))))

```

2026-09-18 18:53:23.132215
 2026-09-18 18:53:23.132384
 169.00000μ 0.000169

In [459]:

```

1 dt1 = datetime.datetime.now()
2 print(dt1)
3 startOfDay = datetime.datetime.fromisoformat(datetime.date.today().isoformat())
4 print(startOfDay)
5 td = dt1 - startOfDay
6 print(td.total_seconds())

```

2026-09-18 18:57:44.890444
 2026-09-18 00:00:00
 68264.890444

In [461]:

```

1 td = datetime.timedelta(weeks = 52, seconds = 86400)
2 print(datetime.datetime.now() + td)

```

2027-09-18 19:01:37.580957

In [467]:

```

1 print(time.strftime("%FT%T", time.strptime("1/6/2024 2:30 PM", "%d/%m/%Y %I:%M%p")))

```

2024-06-01T14:30:00

In [472]:

```

1 td = datetime.timedelta(days = -1, seconds = 5400)
2 print(td + datetime.datetime.now())
3 print(td.total_seconds() // 60 / 60)

```

2026-09-18 18:41:55.925042
 -22.5

In [480]:

```

1 myTZ = datetime.timezone(datetime.timedelta(hours = 5.5), name = "IST")
2 print(myTZ)
3 UTCTZ = datetime.timezone.utc
4 print(UTCTZ)

```

IST
UTC

In [489]:

```
1 print(datetime.datetime.now().replace(tzinfo =  
datetime.timezone(datetime.timedelta(hours = 5.5), name =  
"IST")).astimezone(datetime.timezone(datetime.timedelta(hours =  
9))).isoformat())
```

2026-09-19T21:28:11.050467+09:00

In [507]:

```
1 currentTime = datetime.datetime.now()  
2 tzIST = datetime.timezone(datetime.timedelta(hours = 5.5))  
3 currentISTTime = currentTime.replace(tzinfo = tzIST)  
4 tzJST = datetime.timezone(datetime.timedelta(hours = 9))  
5 currentJSTTime = currentISTTime.astimezone(tzJST)  
6 print(currentJSTTime)  
7 currentJSTTime = currentJSTTime.replace(tzinfo = None)  
8 tzDifference = currentJSTTime - datetime.datetime.now()  
9 print(tzDifference)  
10 print(type(tzDifference).      )  
11 print(round(tzDifference.total_seconds() // 60 / 60, 1))
```

2026-09-19 21:49:02.698258+09:00
3:29:59.999810
timedelta
3.5

In [514]:

```
1 epochTime = 12345678.9  
2 dt = datetime.datetime.fromtimestamp(epochTime).replace(day = 1, hour = 0,  
3 minute = 0, second = 0, microsecond = 0)  
print(repr(dt))
```

datetime.datetime(1970, 5, 1, 0, 0)

In []:

```
1 epochTime = 12345678.9  
2 dt = datetime.datetime.fromtimestamp(epochTime)  
3 print(repr(datetime.datetime(year = dt.year, month = dt.month, day = 1)))
```

datetime.datetime(2026, 9, 1, 0, 0)

In []:

```
1 epochTime = time.time()  
2  
3 dt = datetime.datetime.fromtimestamp(epochTime)  
4 print(repr(datetime.datetime(year = dt.year, month = dt.month, day = 1)))
```

datetime.datetime(2026, 9, 1, 0, 0)

In [520]:

```
1 start = "2001-01-01T13:50:23"  
2 end = "2001-06-12T14:23:50"  
3 diff = int(round((datetime.datetime.fromisoformat(end) -  
4 datetime.datetime.fromisoformat(start)).total_seconds() // 60 / 60, 0))  
print(diff)
```

3889

In [544]:

```
1 start = "2001-01-01T00:00:00"
2 end = "2001-01-01T01:00:00"
3 diff = int(round((datetime.datetime.fromisoformat(end) -
4 datetime.datetime.fromisoformat(start)).total_seconds() // 60 / 60, 0))
5 print(diff)
```

1

In [545]:

```
1 start = "2001-01-01T00:00:00"
2 end = "2001-01-02T01:50:00"
3 diff = int(round((datetime.datetime.fromisoformat(end) -
4 datetime.datetime.fromisoformat(start)).total_seconds() // 60 / 60, 0))
5 print(diff)
```

26

In [534]:

```
1 def formatDateTime(dateTimeObj, formatStr = "%FT%T"):
2     return dateTimeObj.strftime(formatStr)
3
4
5 dateTimeObj = datetime.datetime.strptime("02/04/2025 03:30:30 PM", "%m/%d/%Y
6 %I:%M:%S %p")
7 print(formatDateTime(dateTimeObj))
8 print(formatDateTime(dateTimeObj, "%Y-%m-%d"))
```

2025-02-04T15:30:30

2025-02-04

In [546]:

```
1 def formatDateTime(dateTimeObj, formatStr = "%FT%T"):
2     return dateTimeObj.strftime(formatStr)
3
4
5 dateTimeObj = datetime.datetime(2020, 2, 1, 13, 34, 5)
6 print(formatDateTime(dateTimeObj))
7 print(formatDateTime(dateTimeObj, "%Y-%m-%d"))
```

2020-02-01T13:34:05

2020-02-01

In [50]:

```
1 with open("../var/actors.csv") as fh:
2     itr = csv.reader(fh)
3     for row in itr:
4         print(*row)
```

First Name Last Name DOB Sketches

John Cleese 10/27/39 The Cheese Shop, Ministry of Silly Walks, It's the Arts

Eric Idle 3/29/43 The Cheese Shop, Nudge Nudge, "Spam"

Peter O'Toole 8/2/32 Lawrence of Arabia

In []:

```
1 with open("../var/actors.pdv") as fh:
2     itr = csv.reader(fh, delimiter = "|", quotechar = "'", escapechar =
3     "\\ ", skipinitialspace = True)
4     for row in itr:
5         print(row)
```

```
['FIRST_NAME', 'LAST_NAME', 'DOB', 'SKETCHES']  
['John', 'Cleese', '10/27/39', "The Cheese Shop, Ministry of Silly Walks, It's the Arts"]  
['Eric', 'Idle', '3/29/43', 'The Cheese Shop, Nudge Nudge, "Spam"']  
['Peter', "O'Toole", '8/2/32', 'Lawrence of Arabia']
```

```
In [14]: 1 csv.register_dialect("pdv", delimiter = "|", quotechar = "'", skipinitialspace  
2         = True, escapechar = "\\")  
        csv.list_dialects()
```

```
Out[14]: ['excel', 'excel-tab', 'unix', 'mine', 'pdv']
```

```
In [563]: 1 with open("../var/actors.pdv") as fh:  
2         itr = csv.reader(fh, dialect = "pdv")  
3         for row in itr:  
4             print(row)
```

```
['FIRST_NAME', 'LAST_NAME', 'DOB', 'SKETCHES']  
['John', 'Cleese', '10/27/39', "The Cheese Shop, Ministry of Silly Walks, It's the Arts"]  
['Eric', 'Idle', '3/29/43', 'The Cheese Shop, Nudge Nudge, "Spam"']  
['Peter', "O'Toole", '8/2/32', 'Lawrence of Arabia']
```

```
In [13]: 1 csv.register_dialect("mine", delimiter = "|", skipinitialspace = True,  
2         escapechar = "\\ ", doublequote = False)  
        csv.list_dialects()
```

```
Out[13]: ['excel', 'excel-tab', 'unix', 'mine']
```

```
In [56]: # csv.register_dialect("mine", delimiter = "|", skipinitialspace = True,  
1         quoting = csv.QUOTE_NONE, escapechar = "\\ ", doublequote = False,  
2         lineterminator = "\n")  
2 csv.register_dialect("mine", delimiter = "|", skipinitialspace = True, quoting  
3 = csv.QUOTE_NONE, lineterminator = "\n", escapechar = "\\ ", doublequote =  
False, quotechar = None)  
        csv.list_dialects()
```

```
Out[56]: ['excel', 'excel-tab', 'unix', 'mine', 'pdv']
```

In [59]:

```
1 print("Current literal contents...")
2 with open("../var/actors.pdv") as fh1:
3     for ln in fh1:
4         print(ln, end = "")
5
6 print("\n\nAfter converting to my format...")
7 with open("../var/actors.pdv") as fh1:
8     with open("../var/actors.tsv", "w") as fh2:
9         itr1 = csv.reader(fh1, dialect = "pdv")
10        itr2 = csv.writer(fh2, dialect = "mine")
11        for row in itr1:
12            itr2.writerow(row)
13
14 with open("../var/actors.tsv") as fh:
15     for row in fh:
16         print(row, end = "")
```

Current literal contents...

FIRST_NAME| LAST_NAME| DOB| SKETCHES

John|Cleese| 10/27/39| 'The Cheese Shop, Ministry of Silly Walks, It\'s the Arts'

Eric| Idle| 3/29/43| 'The Cheese Shop, Nudge Nudge, "Spam"'

Peter| 'O\'Toole'| 8/2/32| Lawrence of Arabia

After converting to my format...

FIRST_NAME|LAST_NAME|DOB|SKETCHES

John|Cleese|10/27/39|The Cheese Shop, Ministry of Silly Walks, It's the Arts

Eric|Idle|3/29/43|The Cheese Shop, Nudge Nudge, "Spam"

Peter|O'Toole|8/2/32|Lawrence of Arabia

In [566]:

```
1 with open("../var/file1.csv") as fh:
2     for ln in fh:
3         print(ln, end = "")
```

col1,col2,col3

10,20,30

30,40,50

60,70,80

In [571]:

```
1 with open("../var/file1.csv") as fh:
2     itr = csv.reader(fh)
3     smallest, largest = math.inf, -math.inf
4     for row in itr:
5         for fld in row:
6             try:
7                 n = int(fld)
8             except ValueError:
9                 pass
10            if n < smallest:
11                smallest = n
12            if n > largest:
13                largest = n
14
15 print(smallest, largest)
```

10 80

```
In [572]: 1 with open("../var/file2.csv") as fh:
          2     for ln in fh:
          3         print(ln, end = "")
```

```
1, 3.14, 500
20, 1, -2
-1.1, -2.2, -3.3
```

```
In [588]: 1 print(type(2.0))
          2 print(type(2.0) == float)
          3 print(isinstance(2.0, float))
```

```
<class 'float'>
True
True
```

```
In [609]: 1 a = "500.0"
          2 b = "500"
          3 if "." in a:
          4     print("a is fractional")
          5 if "." in b:
          6     print("b is fractional")
```

```
a is fractional
```

```
In [611]: 1 with open("../var/file2.csv") as fh:
          2     itr = csv.reader(fh, skipinitialspace = True)
          3     smallest, largest = math.inf, -math.inf
          4     for row in itr:
          5         for fld in row:
          6             n = float(fld) if "." in fld else int(fld)
          7             if n < smallest:
          8                 smallest = n
          9             if n > largest:
          10                 largest = n
          11
          12 print(smallest, largest)
```

```
-3.3 500
```

```
In [614]: 1 with open("../var/file3.tsv") as fh:
          2     for ln in fh:
          3         print(ln, end = "")
```

```
col1    col2    col3
10       20       30
40       50       60
100      200      300
```

```
In [623]: 1 with open("../var/file3.tsv") as fh:
          2     itr = csv.reader(fh, skipinitialspace = True, delimiter = "\t")
          3     smallest, largest = math.inf, -math.inf
          4     next(fh)
```

```

5         for row in itr:
6             for fld in row:
7                 n = int(fld)
8                 if n < smallest:
9                     smallest = n
10                if n > largest:
11                    largest = n
12
13    print(smallest, largest)

```

10 300

```

In [627]: 1 with open("../var/file4.csv") as fh:
2           for ln in fh:
3               print(ln, end = "")

```

```

-col1-|-col2-|-col3-
10|20|30
-3.14-|-25-|-100-
---3.14-|20|-30-

```

```

In [633]: 1 a = "--3.3-"
2           print(a.removesuffix("-").removeprefix("-"))

```

-3.3

```

In [ ]: 1 print(a.replace("-", ""))

```

3.3

```

In [642]: 1 with open("../var/file4.csv") as fh:
2           itr = csv.reader(fh, delimiter = "|")
3           smallest, largest = math.inf, -math.inf
4           next(fh)
5           for row in itr:
6               for fld in row:
7                   fld = fld.replace("-", "")
8                   n = float(fld) if "." in fld else int(fld)
9                   if n < smallest:
10                       smallest = n
11                   if n > largest:
12                       largest = n
13
14    print(smallest, largest)

```

3.14 100

```

In [648]: 1 a = "b"
2           print(ord(a) - ord("a"))

```

1

```

In [655]: 1 data = [{'a': '1_a', 'b': '1_b', 'c': '1_c'}, {'c': '2_c', 'd': '2_d'}, {'a':
                '3_a', 'c': '3_c', 'e': '3_e'}]
           with open("../var/out2.csv", "w") as fh:

```

```

2      itr = csv.writer(fh)
      itr.writerow("abcde")
3      for d in data:
          row = ["", "", "", "", ""]
4          for k, v in d.items():
              row[ord(k) - ord("a")] = v
5          itr.writerow(row)

6  with open("../var/out2.csv") as fh:
      for ln in fh:
7          print(ln, end = "")
8
9
10
11
12
13

```

```

a,b,c,d,e
1_a,1_b,1_c,,
,,2_c,2_d,
3_a,,3_c,,3_e

```

```

In [8]: 1  for _ in range(10):
2          print(random.random(), end = " ")
3
4  randList = [random.random() for _ in range(5000000)]
5  print("Min:", toEnggNotation(min(randList)))
6  print("Max:", toEnggNotation(max(randList)))
7  minima = min(randList)
8  print(f"{minima:.200f}".rstrip("0"))
9  dec = decimal.Decimal(minima)
10 print(dec)
11 print(dec.normalize())
12 recurring = 1 / 3
13 print(decimal.Decimal(recurring))
14 print(f"{recurring:.200f}")

```

[illegible]

```
In [9]: 1 print(randList[:5])
```

[0.08172163591676762, 0.6379410884560406, 0.36610121100531756, 0.47189283950477456, 0.7181724643263364]

```
In [ ]: 1 recurring = 1 / 10
        2 print(decimal.Decimal(recurring))
        3 print(f"{recurring:.200f}")
```

[illegible]

```
In [89]: 1 recurring = 2000 / 10000
          2 print(f"{recurring:.200f}")
```

[illegible]

```
In [14]: 1 print(f"{random.random():f}")
2 print(random.random())
3 try:
4     print(f"{random.random():b}")
5 except ValueError as ex:
6     print(type(ex).          + ": " + ex.args[0])
7 print(f"{123:b}")
8 print(sys.float_info)
9 try:
10     print(f"{0.1:b}")
11 except ValueError as ex:
12     print(type(ex).          + ": " + ex.args[0])
13 print(int("0b1001", 2))
14 print(int("1001", 2))
15 try:
16     print(int(1001, 2))
17 except TypeError as ex:
18     print(type(ex).          + ": " + ex.args[0])
```

```
1 print(f"{smallestFloat:.1075f}")
2 print(smallestFloat)
```

5e-324

0.0

None

2720295377534757.0 673366308862969.4 5999141237517758.0 6442052165548231.0 1752782305927253.0

```
abc
  abc
..abc
abc  de
abc  de
    abcde
```

```
1
[2, 4, 3]
```

```
7 print(random.choices({0, 1, 2, 3, 4}, k = 3))
```

```

8 except TypeError as ex:
9     print(type(ex).          + ": " + ex.args[0])
10
11 try:
12     print({4, 3, 2, 1, 0}[2])
13 except TypeError as ex:
14     print(type(ex).          + ": " + ex.args[0])

```

```

TypeError: 'set' object is not subscriptable
TypeError: 'set' object is not subscriptable
TypeError: 'set' object is not subscriptable
<>:12: SyntaxWarning: 'set' object is not subscriptable; perhaps you missed a comma?
<>:12: SyntaxWarning: 'set' object is not subscriptable; perhaps you missed a comma?
/tmp/ipykernel_31787/1848623714.py:12: SyntaxWarning: 'set' object is not subscriptable; perhaps you missed a comma?
print({4, 3, 2, 1, 0}[2])

```

```

In [35]: 1 try:
2         print(random.choice({4, 3, 2, 1}))
3     except TypeError as ex:
4         print(type(ex).          + ": " + ex.args[0])

```

```

TypeError: 'set' object is not subscriptable

```

```

In [37]: 1 face_values = ['9', '10', 'J', 'Q', 'K', 'A']
2
3
4 def make_sample_space(face_values):
5     return [(e1, e2) for e1 in face_values for e2 in face_values]
6
7
8 print(make_sample_space(face_values))

```

```

[('9', '9'), ('9', '10'), ('9', 'J'), ('9', 'Q'), ('9', 'K'), ('9', 'A'), ('10', '9'), ('10', '10'),
('10', 'J'), ('10', 'Q'), ('10', 'K'), ('10', 'A'), ('J', '9'), ('J', '10'), ('J', 'J'), ('J', 'Q'),
('J', 'K'), ('J', 'A'), ('Q', '9'), ('Q', '10'), ('Q', 'J'), ('Q', 'Q'), ('Q', 'K'), ('Q', 'A'),
('K', '9'), ('K', '10'), ('K', 'J'), ('K', 'Q'), ('K', 'K'), ('K', 'A'), ('A', '9'), ('A', '10'),
('A', 'J'), ('A', 'Q'), ('A', 'K'), ('A', 'A')]

```

```

In [41]: 1 sample_space = make_sample_space(face_values)
2
3
4 def simulate_throws_from_sample_space(sample_space, n):
5     return [random.choice(sample_space) for _ in range(n)]
6
7
8 print(simulate_throws_from_sample_space(sample_space, 10))

```

```

[('9', 'K'), ('Q', '9'), ('Q', 'Q'), ('J', 'A'), ('A', 'A'), ('A', 'J'), ('A', 'K'), ('K', 'A'),
('10', 'Q'), ('A', 'K')]

```

```

In [43]: 1 def simulate_throws(face_values, n):
2         return [tuple(random.choices(face_values, k = 2)) for _ in range(n)]
3

```

```
4
5 print(simulate_throws(face_values, 10))
```

```
[('Q', 'A'), ('K', 'Q'), ('10', '10'), ('10', '9'), ('Q', 'Q'), ('A', 'K'), ('A', 'K'), ('A', 'Q'), ('10', 'J'), ('10', 'Q')]
```

In []:

```
1
2
3 def countOccurrences(simulationFunction, nThrows):
4     input = sample_space if simulationFunction ==
5     simulate_throws_from_sample_space else face_values
6     throws = simulationFunction(input, nThrows)
7     occurrences = dict.fromkeys(face_values, 0)
8     for k, v in throws:
9         occurrences[k] = occurrences.get(k, 0) + 1
10        occurrences[v] = occurrences.get(v, 0) + 1
11    return occurrences
12
13 print(countOccurrences(simulate_throws, 10))
14 print(countOccurrences(simulate_throws_from_sample_space, 10))
15
16
17
18
19
20
```

```
[('K', 'A'), ('J', 'J'), ('Q', '10'), ('Q', '9'), ('K', '10'), ('K', '10'), ('K', 'Q'), ('A', '9'), ('K', '9'), ('10', '10')]
{'9': 3, '10': 5, 'J': 2, 'Q': 3, 'K': 5, 'A': 2}
[('J', 'A'), ('9', '10'), ('10', '9'), ('Q', 'J'), ('9', 'A'), ('10', 'K'), ('K', 'J'), ('A', 'K'), ('10', '9'), ('A', '9')]
{'9': 5, '10': 4, 'J': 3, 'Q': 1, 'K': 3, 'A': 4}
```

In [3]:

```
1 def randomFloat(low, hig):
2     return low + (random.random() * (hig - low))
3
4
5 print(max([randomFloat(2, 4) for _ in range(100_000_000)]))
```

```
3.9999999851096346
```

In []:

```
1 def randomFloat(low, hig):
2     return low + (random.random() * (hig - low))
3
4
5 print(max([randomFloat(2, 4) for _ in range(100_000_000)]))
```

```
3.9999999851096346
```